

AINSHAMS UNIVERSITY

Faculty of Engineering

International Credit Hours Engineering Programs



Cinema Ticket Booking System

Submitted to:

Dr. Nivin Atef Helal Amin

Eng. Esraa Karam

Submitted by:

Team No. (2)

Farah Mohamed Nawar	24P0377
Farida Khaled ElGanzoury	24P0426
Fatema Alzahraa Mohamed Abdelmaboud	24P0160
Heba Hassan Badawy	24P0481
Hla Mohamed Shebl Sakr	24P0406
Malak Moustafa Ali Abdelhadi	24P0239
Nour Mostafa Abdelaty Melegy	24P0176

1 May 2026

Table of Contents

1. SQL Script	8
1.1 Tables creation	8
1.1.1 Payment Table	8
1.1.2 BOOKING TABLE	8
1.1.3 Seat Table	8
1.1.4 Ticket Table	8
1.1.5 Movie Table	9
1.1.6 Movie Genre Table	10
1.1.7 Hall Table	11
1.1.8 Admin Table	12
1.1.9 CustomerFinal Table	13
1.1.10 Customer Phone Final Table	14
1.1.11 Show Table	15
1.1.12 Transactions Table	17
1.2 Stored Procedures & Functions	18
1.2.1 Cancel Booking (Stored Procedure)	18
1.2.2 Update Payment Method (Stored Procedure)	18
1.2.3 Get Daily Revenue (Scalar Function)	19
1.2.4 Get Payment and Booking Status (Scalar Function)	19
1.2.5 Assign Seat to Ticket (Stored Procedure)	20
1.2.6 Update Show Price by Percentage (Stored Procedure)	20
1.2.7 Add New Movie (Stored Procedure)	21
1.2.8 Get Movies by Genre (Stored Procedure)	21
1.2.9 Add Genre to a Movie (Stored Procedure)	21
1.2.10 Check Hall Capacity (Scalar Function)	22
1.2.11 AddCustomerFinal (Stored Procedure)	22
1.2.12 AddCustomerPhone (Stored Procedure)	23
1.2.13 DeleteCustomer (Stored Procedure)	23
1.2.14 DeleteCustomerPhone (Stored Procedure)	23
1.2.15 CustomerLoginWithDetails (Stored Procedure)	24

1.2.16 log_new_transaction (Stored Procedure).....	24
1.2.17 GetCustomerPhones (Stored Procedure)	25
1.2.18 get_transaction_status (Stored Procedure)	25
1.2.19 getMoviesInHall (Stored Procedure).....	26
1.2.20 DELETE_Scheduled_Show (Stored Procedure).....	26
1.2.21 SearchMovie (Stored Procedure)	27
1.2.22 SP_Schedule_show (Stored Procedure)	27
1.3 Queries	29
1.3.1 Seat Availability.....	29
1.3.2 Movies by Genre	29
2. GUI and Database Interface:.....	31
2.1 User Form	31
2.2 Movie Selection Form	32
2.3 Customer Interface	33
2.3.1 Customer Options	33
2.3.2 Log in Form.....	34
2.3.3 Register Form	35
2.3.4 Customer Dashboard Form	37
2.3.5 Booking Form	39
2.4 Admin Interface	41
2.4.1 Admin Log in Form	41
2.4.2 Admin Dashboard	42
.....	42
2.4.3 Movie Management Dashboard.....	43
2.4.4 Scheduling Dashboard	45
2.4.5 Transaction Interface	48
2.4.6 Hall Management Dashboard.....	49
Appendix	53
ERD.....	53
Schema	54
1 NF	54

2 NF	55
3 NF	56
All Queries Used in Booking Form	57
Booking Page Flow Summary	57
1. Load Show Times.....	57
2. Load Available Seats.....	57
3. Get Show Info & Price (Proceed to Payment)	58
4. Check Hall Capacity	58
5. Count Already Booked Tickets	58
6. Create Booking.....	58
7. Insert Ticket Per Seat	58
8. Confirm Booking.....	58
9. View Ticket Details.....	59
10. Load All Customer Bookings	59
11. Cancel Booking	59
12. Update Payment Method	59
13. Check Booking Status	59

1. SQL Script

1.1 Tables creation

1.1.1 Payment Table

Table Creation

```
CREATE TABLE [payment] (  
    [payment_id] INT IDENTITY(1,1) PRIMARY KEY,  
    [Payment_Method] NVARCHAR(50) CHECK (Payment_Method IN  
( 'Visa', 'cash', 'MasterCard', 'Fawry', 'Vodafone Cash', 'InstaPay', 'Apple Pay' )),  
    [Amount] MONEY,  
    Payment_status NCHAR(15) CHECK (Payment_status IN  
( 'Completed', 'Pending', 'Canceled' ))  
);
```

Description: Stores payment records for each booking. Each payment has a unique ID, a constrain to accepted payment options, a monetary amount, and a status tracking whether the payment was completed, pending, or canceled.

Constraints:

- PRIMARY KEY on payment_id (IDENTITY(1,1))
- CHECK on Payment_Method — restricts to 7 accepted methods
- CHECK on Payment_status — restricts to 3 valid statuses

Sample Data Insertion

```
INSERT INTO payment (Payment_Method, Amount, Payment_status)  
VALUES  
( 'Visa', 300, 'Pending'),  
( 'cash', 450, 'Completed'),  
( 'MasterCard', 450, 'Completed'),  
( 'cash', 120, 'Completed'),  
( 'Fawry', 200, 'Pending'),  
( 'Visa', 600, 'Canceled'),  
( 'Vodafone Cash', 350, 'Completed'),  
( 'MasterCard', 1000, 'Pending'),  
( 'InstaPay', 280, 'Completed'),  
( 'Apple Pay', 150, 'Completed');
```

1.1.2 BOOKING TABLE

Table Creation

```
CREATE TABLE [Booking] (  
    [Booking_ID] IDENTITY(1,1) INT PRIMARY KEY,  
    [Booking_Status] NVARCHAR(15) CHECK (Booking_Status IN ('Confirmed','Pending','Canceled')),  
    [Booking_date] DATETIME,  
    [B_shownumber] INT ,  
    [Customer_ID] INT ,  
    [Payment_ID] INT UNIQUE,  
    CONSTRAINT FK_Booking_Show FOREIGN KEY ([B_shownumber]) REFERENCES [Show]([show_Number]) ON DELETE CASCADE ON  
UPDATE CASCADE,  
    CONSTRAINT FK_Booking_Customer FOREIGN KEY ([Customer_ID]) REFERENCES [Customerfinal]([Customer_ID]) ON DELETE  
CASCADE ON UPDATE CASCADE,  
    CONSTRAINT FK_Booking_Payment FOREIGN KEY ([Payment_ID]) REFERENCES [payment]([payment_id])  
ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Description: The central transactional table linking customers, shows, and payments. Each booking belongs to one customer and one show, and is tied to exactly one payment record. The UNIQUE constraint on Payment_ID enforces a one-to-one relationship between a booking and a payment.

Constraints:

- PRIMARY KEY on Booking_ID (IDENTITY(1,1))
- CHECK on Booking_Status
- UNIQUE on Payment_ID — one payment per booking
- Three FOREIGN KEY constraints linking to Show, Customerfinal, and payment

Sample Data Insertion

```
INSERT INTO Booking (Booking_Status, Booking_date, B_shownumber, Customer_ID,  
Payment_ID)
```

VALUES

```
('Pending', '2026-04-07 10:00:00', 1, 1, 1),  
('Confirmed', '2026-04-07 11:30:00', 2, 2, 2),  
('Confirmed', '2026-04-07 12:45:00', 3, 3, 3),  
('Pending', '2026-04-08 09:15:00', 4, 4, 4),  
('Canceled', '2026-04-08 14:00:00', 5, 5, 5),  
('Confirmed', '2026-04-09 10:30:00', 6, 6, 6),  
('Pending', '2026-04-09 15:20:00', 7, 7, 7),  
('Confirmed', '2026-04-10 11:00:00', 8, 8, 8),  
('Confirmed', '2026-04-10 13:45:00', 9, 9, 9),  
('Pending', '2026-04-11 08:30:00', 10, 10, 10);
```


1.1.3 Seat Table

Table Creation

```
CREATE TABLE Seat (  
  HNumber      INT NOT NULL,  
  SeatNumber   INT NOT NULL,  
  SeatArrangement VARCHAR(50) CHECK (SeatArrangement IN (  
    'Front-Right', 'Front-Left', 'Front-Center',  
    'Back-Right', 'Back-Left', 'Back-Center'  
  )),  
  PRIMARY KEY (HNumber, SeatNumber),  
  FOREIGN KEY (HNumber) REFERENCES Hall(HallNumber)  
  
ON DELETE CASCADE  
  
ON UPDATE CASCADE  
);
```

Description: Stores individual seats within each hall. The composite primary key (HNumber, SeatNumber) ensures a seat is defined by a certain hall and its own number inside that hall. Each seat has a physical arrangement label describing its position in the hall.

Constraints:

- Composite PRIMARY KEY on (HNumber, SeatNumber)
- CHECK on SeatArrangement — 6 valid positions
- FOREIGN KEY on HNumber referencing Hall

Sample Data Insertion

```
INSERT INTO Seat (HNumber, SeatNumber, SeatArrangement)  
VALUES  
(1, 1, 'Front-Right'),  
(1, 2, 'Front-Left'),  
(1, 3, 'Front-Center'),  
(2, 1, 'Back-Right'),  
(2, 2, 'Back-Left'),  
(2, 3, 'Back-Center'),  
(3, 1, 'Front-Right'),  
(3, 2, 'Front-Left'),  
(3, 3, 'Back-Right'),  
(4, 1, 'Back-Center');
```


1.1.4 Ticket Table

Table Creation

```
CREATE TABLE Ticket (  
    TicketID          INT IDENTITY(1,1) PRIMARY KEY,  
    TicketStatus      VARCHAR(50) DEFAULT 'Unbooked' CHECK (TicketStatus IN  
( 'Booked', 'Unbooked' )),  
    NumberOfTickets  INT NOT NULL,  
    BookingID        INT NOT NULL,  
    HNumber          INT NOT NULL,  
    Seatnumber       INT NOT NULL,  
    FOREIGN KEY (BookingID) REFERENCES Booking(Booking_ID) ON DELETE CASCADE,  
    FOREIGN KEY (HNumber, Seatnumber) REFERENCES Seat(HNumber, SeatNumber),  
    UNIQUE (HNumber, SeatNumber)  
);
```

Description: Represents the physical ticket issued for a booking, linked to a specific seat in a specific hall. The UNIQUE constraint on (HNumber, SeatNumber) ensures no two tickets are assigned the same seat, preventing double-booking of seats. The NumberOfTickets field records the quantity associated with each ticket record.

Constraints:

- PRIMARY KEY on TicketID (IDENTITY(1,1))
- DEFAULT 'Unbooked' with CHECK on TicketStatus
- FOREIGN KEY on BookingID referencing Booking — many tickets can belong to one booking
- UNIQUE on (HNumber, SeatNumber) — one ticket per seat
- FOREIGN KEY on (HNumber, SeatNumber) referencing Seat

Sample Data Insertion

```
INSERT INTO Ticket (TicketStatus, NumberOfTickets, BookingID, HNumber,  
Seatnumber)  
VALUES  
( 'Booked',    2, 1,    1, 1),  
( 'Unbooked',  1, 2,    1, 2),  
( 'Booked',    3, 3,    1, 3),  
( 'Booked',    2, 4,    2, 1),  
( 'Unbooked',  4, 5,    2, 2),  
( 'Booked',    1, 6,    2, 3),  
( 'Booked',    2, 7,    3, 1),  
( 'Booked',    5, 8,    3, 2),
```

```
('Unbooked', 2, 9, 3, 3),  
( 'Booked', 1, 10, 4, 1);
```

1.1.5 Movie Table

Table Creation

```
CREATE TABLE Movie (  
    EIDR      VARCHAR(20) PRIMARY KEY,  
    Title     VARCHAR(100) NOT NULL,  
    Duration  INT NOT NULL,  
    AgeRating VARCHAR(10) NOT NULL,  
    CONSTRAINT CK_Duration CHECK (Duration > 0),  
    CONSTRAINT CK_AgeRating CHECK (AgeRating IN ('G','PG','PG-13','R'))  
);
```

Description: Stores the core details of each movie. The EIDR (Entertainment Identifier Registry) serves as a unique identifier. Duration must be a positive integer (minutes), and age rating is constrained to standard classification values.

Constraints:

- PRIMARY KEY on EIDR
- NOT NULL on Title, Duration, AgeRating
- Named CHECK constraint CK_Duration — duration must be > 0
- Named CHECK constraint CK_AgeRating — 4 valid rating values

Sample Data Insertion

```
INSERT INTO Movie (EIDR, Title, Duration, AgeRating)  
VALUES  
( 'EIDR-1', 'The Beginning',    120, 'PG'),  
( 'EIDR-2', 'Dark Waters',      95, 'PG-13'),  
( 'EIDR-3', 'Lost in Time',    110, 'G'),  
( 'EIDR-4', 'The Last Stand',  130, 'R'),  
( 'EIDR-5', 'Rising Sun',      105, 'PG'),  
( 'EIDR-6', 'Midnight Run',     88, 'PG-13'),  
( 'EIDR-7', 'Shadow Falls',    115, 'PG-13'),  
( 'EIDR-8', 'Broken Wings',    100, 'R'),  
( 'EIDR-9', 'Edge of Tomorrow', 125, 'PG-13'),  
( 'EIDR-10', 'Final Horizon',  140, 'R');
```

1.1.6 Movie Genre Table

Table Creation

```
CREATE TABLE Movie_Genre (  
    EIDR VARCHAR(20),  
    Genre VARCHAR(50) NOT NULL,  
    PRIMARY KEY (EIDR, Genre),  
    FOREIGN KEY (EIDR) REFERENCES Movie(EIDR) ON DELETE CASCADE ON UPDATE  
CASCADE,  
    CONSTRAINT CK_Genre CHECK (Genre IN (  
        'Action', 'Comedy', 'Drama', 'Horror', 'Romance',  
        'Thriller', 'Sci-Fi', 'Animation', 'Adventure', 'Documentary'  
    ))  
);
```

Description: Implements a many-to-many relationship between movies and genres, allowing each movie to belong to multiple genres. The composite primary key prevents duplicate genre assignments for the same movie. Cascade rules ensure genre records are automatically removed or updated when the parent movie is deleted or its EIDR changes.

Constraints:

- Composite PRIMARY KEY on (EIDR, Genre)
- FOREIGN KEY on EIDR with ON DELETE CASCADE ON UPDATE CASCADE
- CHECK on Genre — constrained to 10 accepted genre values

Sample Data Insertion

```
INSERT INTO Movie_Genre (EIDR, Genre)  
VALUES  
( 'EIDR-1', 'Action'),  
( 'EIDR-1', 'Sci-Fi'),  
( 'EIDR-2', 'Thriller'),  
( 'EIDR-2', 'Action'),  
( 'EIDR-3', 'Romance'),  
( 'EIDR-3', 'Drama'),  
( 'EIDR-4', 'Animation'),  
( 'EIDR-4', 'Comedy'),  
( 'EIDR-5', 'Horror'),  
( 'EIDR-6', 'Adventure');
```

1.1.7 Hall Table

Table creation

```
create table Hall (  
HallNumber int NOT NULL IDENTITY(1,1) primary key,  
HallCapacity int NOT NULL CHECK (HallCapacity between 20 and 50),  
HallType VARCHAR(20) NOT NULL CHECK(HallType in('VIP','Standard'))  
);
```

Description: Stores information about cinema halls. Each hall is uniquely identified by its hall number and includes details about its seating capacity and type. The hall type distinguishes between VIP and Standard halls, while the capacity is restricted to a realistic range to ensure valid data entry.

Constrain:

- PRIMARY KEY on HallNumber IDENTITY(1,1)
- NOT NULL on all attributes
- CHECK on HallCapacity — must be between 20 and 50 seats
- CHECK on HallType — restricts values to ('VIP', 'Standard')

Sample Data Insertion

```
INSERT INTO Hall ( HallCapacity, HallType)  
VALUES  
( 20, 'VIP'),  
( 25, 'Standard'),  
( 30, 'VIP'),  
( 35, 'Standard'),  
( 40, 'VIP'),  
(45, 'Standard'),  
( 50, 'VIP'),  
(30, 'Standard'),  
( 25, 'VIP'),  
( 20, 'Standard');
```

1.1.8 Admin Table

Table creation

```
create table admin (  
admin_id INT IDENTITY(1,1) primary key,  
admin_email nvarchar(100) unique not null check (admin_email like '%_@_%._%'),  
admin_password nvarchar(100) not null,  
admin_fname nvarchar(50) not null,  
admin_lname nvarchar(50) not null  
);
```

Description: Stores administrator's account information for the system. Each admin is uniquely identified by an auto-generated ID and has a unique email used for identification and login purposes. The table also stores the administrator's first and last name along with their password.

Constrain:

- PRIMARY KEY on admin_id (IDENTITY(1,1))
- UNIQUE constraint on admin_email — prevents duplicate admin accounts
- NOT NULL on all attributes — ensures complete admin records

Sample Data Insertion

```
INSERT INTO admin ( admin_email, admin_password, admin_fname, admin_lname)  
VALUES  
( 'eyanadmin1@gmail.com', 'admin123', 'Eyan', 'Doe'),  
( 'johnadmin2@gmail.com', 'admin456', 'John', 'Smith'),  
( 'saraadmin3@gmail.com', 'admin789', 'Sara', 'Wayn');
```


1.1.9 CustomerFinal Table

Table creation

```
CREATE TABLE Customerfinal (  
    Customer_ID INT IDENTITY(1,1) PRIMARY KEY,  
    FirstName NVARCHAR(50) NOT NULL,  
    LastName NVARCHAR(50) NOT NULL,  
    Email NVARCHAR(100) NOT NULL UNIQUE,  
    Cus_Password NVARCHAR(10) NOT NULL,  
    CONSTRAINT CHK_Customer_Email  
    CHECK (Email LIKE '%_@_%._%')  
);
```

Description: Stores customer's account information for the system. Each customer is assigned a unique auto-generated ID and must provide personal details such as first name, last name, email, and password. The email field is enforced to be unique and must follow a valid format, ensuring proper identification and login functionality.

Constrain:

- PRIMARY KEY on Customer_ID (IDENTITY(1,1))
- NOT NULL on all attributes
- UNIQUE on Email — prevents duplicate customer accounts
- CHECK constraint CHK_Customer_Email — ensures email follows a valid format pattern

Sample Data Insertion

```
INSERT INTO Customerfinal (FirstName, LastName, Email, Cus_Password)  
VALUES  
( 'Ali', 'Ahmed', 'ali@gmail.com', 'ali123'),  
( 'Sara', 'Mohamed', 'sara@gmail.com', 'sara123'),  
( 'Omar', 'Khaled', 'omar@gmail.com', 'omar123'),  
( 'Mona', 'Adel', 'mona@gmail.com', 'mona123'),  
( 'Youssef', 'Hassan', 'youssef@gmail.com', 'yousef12'),  
( 'Nour', 'Ali', 'nour@gmail.com', 'nour123'),  
( 'Hassan', 'Tarek', 'hassan@gmail.com', 'hassan12'),  
( 'Laila', 'Samir', 'laila@gmail.com', 'laila123'),  
( 'Karim', 'Mostafa', 'karim@gmail.com', 'karim123'),  
( 'Dina', 'Wael', 'dina@gmail.com', 'dina123');
```

1.1.10 Customer Phone Final Table

Table creation

```
CREATE TABLE CustomerPhonefinal (  
    CustomerID INT NOT NULL,  
    PhoneNo NVARCHAR(20) NOT NULL unique,  
    CONSTRAINT PKCustomerPhone  
    PRIMARY KEY (CustomerID, PhoneNo),  
    CONSTRAINT CHKCustomerPhone CHECK (PhoneNo LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'),  
    CONSTRAINT FK_CustomerPhone_Customer FOREIGN KEY (CustomerID) REFERENCES Customerfinal(Customer_ID)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE );
```

Description: Stores phone numbers associated with customers. This table allows each customer to have multiple phone numbers, while ensuring that each phone number is unique across the system. The composite primary key ensures that no duplicate phone numbers are assigned to the same customer. Cascade rules ensure that phone records are automatically updated or deleted when the related customer record changes.

Constrain:

- Composite PRIMARY KEY on (CustomerID, PhoneNo)
- UNIQUE on PhoneNo — ensures no duplicate phone numbers across customers
- NOT NULL on all attributes
- CHECK constraint CHKCustomerPhone — ensures phone number is exactly 11 digits
- FOREIGN KEY on CustomerID referencing Customerfinal(Customer_ID)
 - ON DELETE CASCADE — deletes phone numbers when customer is deleted
 - ON UPDATE CASCADE — updates phone numbers when customer ID changes

Sample Data Insertion

```
INSERT INTO CustomerPhonefinal (CustomerID, PhoneNo)  
VALUES  
(1, '01011111111'),  
(2, '01022222222'),  
(3, '01033333333'),  
(4, '01044444444'),  
(5, '01055555555'),  
(6, '01066666666'),  
(7, '01077777777'),  
(8, '01088888888'),  
(9, '01099999999'),  
(10, '01100000000');
```

1.1.11 Show Table

Table creation

```
CREATE TABLE [Show] (  
    [show_Number] INT IDENTITY(1,1) PRIMARY KEY,  
    [Price] DECIMAL(10, 2) NOT NULL,  
    [Showtime] DATETIME NOT NULL,  
    [show_admin_id] INT NOT NULL,  
    [sh_EIDR] VARCHAR(20) NOT NULL,  
    [HallNumber] INT NOT NULL,  
  
    CONSTRAINT CK_Show_Price_Positive  
        CHECK ([Price] > 0),  
  
    CONSTRAINT FK_Show_Admin  
        FOREIGN KEY (show_admin_id)  
        REFERENCES admin(admin_id)  
        ON DELETE NO ACTION  
        ON UPDATE NO ACTION,  
  
    CONSTRAINT FK_Show_Movie  
        FOREIGN KEY (sh_EIDR)  
        REFERENCES Movie(EIDR) ON DELETE CASCADE  
        ON UPDATE CASCADE,  
  
    CONSTRAINT FK_Show_Hall  
        FOREIGN KEY (HallNumber)  
        REFERENCES Hall(HallNumber)  
        ON UPDATE CASCADE  
);
```

Description: Stores information about movie showings in the cinema. Each show represents a scheduled screening of a specific movie in a specific hall at a given time. The table connects three main entities: **Admin** (who scheduled the show), **Movie** (what is being shown), and **Hall** (where it is shown). This makes the Show table a central component in managing cinema operations.

Constraints:

- PRIMARY KEY on show_Number IDENTITY(1,1)
- NOT NULL on all attributes
- CHECK constraint CK_Show_Price_Positive — ensures ticket price is greater than 0
- FOREIGN KEY FK_Show_Admin:
 - References admin(admin_id)
 - ON DELETE NO ACTION — prevents deleting admin if linked to a show
 - ON UPDATE NO ACTION

- FOREIGN KEY FK_Show_Movie:
 - References Movie(EIDR) — links each show to a movie
- FOREIGN KEY FK_Show_Hall:
 - References Hall(HallNumber) — ensures show is assigned to a valid hall

Sample Data Insertion

```
INSERT INTO [Show] ( Price, Showtime, show_admin_id, sh_EIDR, HallNumber)
```

```
VALUES
```

```
( 200.00, '2026-04-07 10:00:00', 1, 'EIDR-1', 1),  
( 300.00, '2026-04-07 11:30:00', 2, 'EIDR-2', 2),  
( 400.00, '2026-04-07 12:45:00', 2, 'EIDR-3', 3),  
( 400.00, '2026-04-08 09:15:00', 1, 'EIDR-4', 4),  
( 200.00, '2026-04-08 14:00:00', 3, 'EIDR-5', 5),  
( 200.00, '2026-04-09 10:30:00', 2, 'EIDR-6', 6),  
( 300.00, '2026-04-09 12:30:00', 3, 'EIDR-7', 7),  
( 400.00, '2026-04-09 16:30:00', 2, 'EIDR-8', 8),  
( 200.00, '2026-04-10 09:15:00', 1, 'EIDR-9', 9),  
( 300.00, '2026-04-10 11:15:00', 3, 'EIDR-10', 10);
```

1.1.12 Transactions Table

Table creation

```
create table transactions (  
    transaction_id INT IDENTITY(1,1) primary key,  
    transaction_datetime datetime not null,  
    transaction_type nvarchar(20) check (transaction_type in ('payment','refund')),  
    transaction_status nvarchar(20) check (transaction_status in('pending','completed','failed')),  
    payment_id int not null,  
    foreign key (payment_id) references payment(payment_id)  
  
ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Description: Stores all financial transactions related to payments, including both payments and refunds. Each transaction is linked to a specific payment record, and includes the transaction date, type, and current status. This table helps track the lifecycle of financial operations in the system.

Constraints:

- PRIMARY KEY on transaction_id (IDENTITY(1,1))
- NOT NULL on transaction_datetime and payment_id
- CHECK on transaction_type — allows only ('payment', 'refund')
- CHECK on transaction_status — allows only ('pending', 'completed', 'failed')
- FOREIGN KEY on payment_id referencing payment(payment_id)

Sample Data Insertion

```
INSERT INTO transactions  
( transaction_datetime, transaction_type, transaction_status, payment_id)  
VALUES  
( '2026-03-01 10:00:00', 'payment', 'completed', 1),  
( '2026-04-02 14:30:00', 'payment', 'pending', 2),  
( '2026-01-03 09:15:00', 'refund', 'failed', 3);
```

1.2 Stored Procedures & Functions

1.2.1 Cancel Booking (Stored Procedure)

```
CREATE PROCEDURE cancel_booking
    @bookingid INT
AS
BEGIN
    DECLARE @PayID INT;
    SELECT @PayID = Payment_ID FROM Booking WHERE Booking_ID = @BookingID;

    BEGIN TRANSACTION;
    BEGIN TRY
        UPDATE Booking SET Booking_Status = 'Canceled' WHERE Booking_ID =
@BookingID;
        UPDATE payment SET Payment_status = 'Canceled' WHERE payment_id =
@PayID;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

Description: Cancels both the booking and its associated payment in a single transaction. If either update fails, the entire operation is rolled back to preserve data consistency.

1.2.2 Update Payment Method (Stored Procedure)

```
CREATE PROCEDURE sp_UpdatePaymentMethod
    @BookingID INT,
    @NewMethod NVARCHAR(50)
AS
BEGIN
    UPDATE payment
    SET Payment_Method = @NewMethod
    FROM payment P
    JOIN Booking B ON P.payment_id = B.Payment_ID
    WHERE B.Booking_ID = @BookingID;
END;
```

Description: Updates the payment method for a booking by joining payment with Booking on the payment ID. Allows changing the payment method (e.g., from cash to Visa) without needing to know the payment ID directly.

1.2.3 Get Daily Revenue (Scalar Function)

```
CREATE FUNCTION fn_getDailyRevenue (@Target_date DATE)
RETURNS MONEY
AS
BEGIN
    DECLARE @total MONEY;
    SELECT @total = SUM(T.NumberOfTickets * S.Price)
    FROM Booking B
    INNER JOIN Ticket T ON B.Booking_ID = T.BookingID
    INNER JOIN [Show] S ON B.B_shownumber = S.show_Number
    WHERE CAST(B.Booking_date AS DATE) = @Target_date
    AND B.Booking_Status = 'Confirmed';
    RETURN ISNULL(@total, 0);
END;
```

Description: Calculates total daily revenue by multiplying each ticket's count by the show price, filtered to confirmed bookings only on the given date. Returns 0 if no confirmed bookings exist for that date.

1.2.4 Get Payment and Booking Status (Scalar Function)

```
CREATE FUNCTION get_payment_status (@BookingID INT)
RETURNS NVARCHAR(30)
AS
BEGIN
    DECLARE @pstatus NVARCHAR(50);
    DECLARE @Bstatus NVARCHAR(50);
    SELECT @pstatus = p.Payment_status, @Bstatus = Booking_Status
    FROM payment p
    JOIN Booking b ON b.Payment_ID = p.payment_id
    WHERE b.Booking_ID = @BookingID;
    RETURN @Bstatus + ' ' + @pstatus;
END;
```

Description: Returns a combined status string (e.g., "Confirmed Completed") for a given booking, joining the booking status from Booking with the payment status from payment.

1.2.5 Assign Seat to Ticket (Stored Procedure)

```
CREATE PROCEDURE AssignSeatToTicket
    @TicketID    INT,
    @HNumber     INT,
    @SeatNumber  INT
AS
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM Seat WHERE HNumber = @HNumber AND SeatNumber = @SeatNumber
    )
        BEGIN PRINT 'Seat does not exist in this hall'; RETURN; END

    IF EXISTS (
        SELECT 1 FROM Ticket
        WHERE HNumber = @HNumber AND SeatNumber = @SeatNumber AND TicketID <>
@TicketID
    )
        BEGIN PRINT 'Seat already booked by another ticket'; RETURN; END

    UPDATE Ticket
    SET HNumber = @HNumber, SeatNumber = @SeatNumber, TicketStatus = 'Booked'
    WHERE TicketID = @TicketID;

    PRINT 'Seat successfully assigned to ticket';
END;
```

Description: Safely assigns a seat to a ticket with two guard checks: first verifies the seat exists in the hall, then verifies it is not already taken by a different ticket. Only if both checks pass does it update the ticket status and mark it as booked.

1.2.6 Update Show Price by Percentage (Stored Procedure)

```
CREATE PROCEDURE UpdateShowPrice
    @ShowNO      VARCHAR(20),
    @PricePercentage DECIMAL(5,2)
AS
BEGIN
    UPDATE [Show]
    SET Price = Price + (Price * @PricePercentage / 100)
    WHERE show_Number = @ShowNO;
END;
```


Description: Increases the price of a given show by a specified percentage. Useful for applying dynamic pricing (e.g., peak-hour surcharges).

1.2.7 Add New Movie (Stored Procedure)

```
CREATE PROCEDURE AddMovie
    @EIDR    VARCHAR(20),
    @Title    VARCHAR(100),
    @Duration INT,
    @AgeRating VARCHAR(10)
AS
BEGIN
    INSERT INTO Movie (EIDR, Title, Duration, AgeRating)
    VALUES (@EIDR, @Title, @Duration, @AgeRating);
END;
```

Description: Inserts a new movie record into the Movie table. All parameters are required and must satisfy the table's constraints.

1.2.8 Get Movies by Genre (Stored Procedure)

```
CREATE PROCEDURE GetMoviesByGenre
    @Genre VARCHAR(50)
AS
BEGIN
    SELECT
        m.EIDR,
        m.Title,
        m.Duration,
        m.AgeRating
    FROM Movie m
    INNER JOIN Movie_Genre mg
        ON m.EIDR = mg.EIDR
    WHERE mg.Genre = @Genre
    ORDER BY m.Title;
END;
```

Description: Returns all movies belonging to a specified genre by joining Movie and Movie_Genre. Useful for filtering the catalog by genre on the front end.

1.2.9 Add Genre to a Movie (Stored Procedure)

```
CREATE PROCEDURE AddGenreToMovie
    @MovieEIDR VARCHAR(20),
    @Genre      VARCHAR(50)
AS
BEGIN
    INSERT INTO Movie_Genre (EIDR, Genre)
    VALUES (@MovieEIDR, @Genre);
END;
```

Description: Adds a new genre association to an existing movie. The inserted genre must pass the CK_Genre check constraint defined on the Movie_Genre table.

1.2.10 Check Hall Capacity (Scalar Function)

```
create Function CheckHallCapacity
    (@HallNumberParameter int)
    Returns int
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Hall WHERE HallNumber = @HallNumberParameter)
        RETURN -1;

    Declare @ReturnHallCapacity int
    select @ReturnHallCapacity = HallCapacity
    from Hall
    where HallNumber = @HallNumberParameter
    return @ReturnHallCapacity;
END;
```

Description: This function returns the seating capacity of a specific hall based on its hall number. It first checks whether the given hall exists in the system. If the hall does not exist, the function returns **-1** as an indicator of an invalid hall number. Otherwise, it retrieves and returns the hall's capacity.

1.2.11 AddCustomerFinal (Stored Procedure)

```
create PROCEDURE AddCustomerfinal

    @FirstName NVARCHAR(50),
```

```

@LastName NVARCHAR(50),
@Email NVARCHAR(100),
@Password NVARCHAR(10)
AS
BEGIN
    INSERT INTO Customerfinal (FirstName, LastName, Email, CUS_Password)
    VALUES ( @FirstName, @LastName, @Email, @Password);
END;

```

Description: This stored procedure is used to register a new customer in the system. It inserts a new record into the Customerfinal table using the provided personal information, including first name, last name, email, and password. The customer ID is automatically generated using the IDENTITY property defined in the table.

1.2.12 AddCustomerPhone (Stored Procedure)

```

CREATE PROCEDURE AddCustomerPhone
    @CustomerID INT,
    @PhoneNo NVARCHAR(20)
AS
BEGIN
    INSERT INTO CustomerPhonefinal (CustomerID, PhoneNo)
    VALUES (@CustomerID, @PhoneNo);
END;

```

Description: This stored procedure is used to add a new phone number for an existing customer. It inserts the provided phone number into the **CustomerPhonefinal** table and links it to the specified customer using the CustomerID.

1.2.13 DeleteCustomer (Stored Procedure)

```

CREATE PROCEDURE DeleteCustomer
    @CustomerID INT
AS
BEGIN
    DELETE FROM Customerfinal
    WHERE Customer_ID = @CustomerID;
END;

```

Description: This stored procedure deletes a customer record from the system based on the provided CustomerID. Due to the ON DELETE CASCADE constraint in related tables (such as CustomerPhonefinal), all associated phone numbers are automatically deleted when the customer is removed.

1.2.14 DeleteCustomerPhone (Stored Procedure)

```
CREATE PROCEDURE DeleteCustomerPhone
    @CustomerID INT,
    @PhoneNo NVARCHAR(20)
AS
BEGIN
    DELETE FROM CustomerPhonefinal
    WHERE CustomerID = @CustomerID
    AND PhoneNo = @PhoneNo;
END;
```

Description: This stored procedure removes a specific phone number associated with a customer. It ensures that only the targeted phone record is deleted without affecting other phone numbers belonging to the same customer.

1.2.15 CustomerLoginWithDetails (Stored Procedure)

```
CREATE PROCEDURE CustomerLoginWithDetails
    @Email NVARCHAR(100),
    @Password NVARCHAR(10)
AS
BEGIN
    SELECT
        c.Customer_ID,
        c.FirstName,
        c.LastName,
        c.Email,
        b.Booking_ID,
        b.Booking_date
    FROM Customerfinal c
    LEFT JOIN Booking b
        ON c.Customer_ID = b.Customer_ID
    WHERE c.Email = @Email
    AND c.CUS_Password = @Password;
END;
```

Description: This stored procedure is used to authenticate a customer and retrieve their personal and booking information after login. It validates the email and password, then returns customer details along with any associated bookings.

1.2.16 log_new_transaction (Stored Procedure)

```
create procedure log_new_transaction
    @transaction_type nvarchar(20),
    @transaction_status nvarchar(20),
    @payment_id int = NULL
as
begin
    IF @transaction_type NOT IN ('payment', 'refund')
BEGIN
    print('Invalid transaction type');
    RETURN;
END
    IF @transaction_status NOT IN ('pending', 'failed', 'completed')
BEGIN
    print('Invalid transaction status');
    RETURN;
END;
    IF @payment_id IS NULL OR NOT EXISTS (SELECT 1 FROM payment WHERE payment_id = @payment_id)
BEGIN
    PRINT 'Invalid payment_id (must exist already)';
    RETURN;
END
    insert into transactions (transaction_datetime, transaction_type, transaction_status, payment_id) values (GETDATE(),
@transaction_type, @transaction_status, @payment_id)
end;
```

Description: This stored procedure records a new transaction in the system. It validates the transaction type, status, and payment ID before inserting the record into the transactions table. It ensures data integrity and prevents invalid transaction entries.

1.2.17 GetCustomerPhones (Stored Procedure)

```
CREATE PROCEDURE GetCustomerPhones
    @CustomerID INT
AS
BEGIN
    SELECT PhoneNo
    FROM CustomerPhonefinal
    WHERE CustomerID = @CustomerID;
END;
```

Description: This stored procedure retrieves all phone numbers associated with a specific customer. It allows the system to display multiple contact numbers linked to a single customer account.

1.2.18 get_transaction_status (Stored Procedure)

```
Create procedure get_transaction_status
@status nvarchar(20)
as
begin
IF @status NOT IN ('pending', 'failed', 'completed')
BEGIN
    print('Invalid transaction status');
    RETURN;
END;
select transaction_id, transaction_datetime, transaction_type, transaction_status, payment_id
from transactions
where transaction_status = @status
end;
```

Description: This stored procedure retrieves all transactions based on their status. It allows filtering transactions to display only pending, completed, or failed records, helping in tracking and monitoring financial operations in the system.

1.2.19 DELETE_Scheduled_Show (Stored Procedure)

```
CREATE PROCEDURE DELETE_Scheduled_show
@SHNumber INT
AS
BEGIN

    -- Check if show exists
    IF NOT EXISTS (SELECT 1 FROM [Show] WHERE show_Number = @SHNumber)
    BEGIN
        PRINT 'Show does not exist.'
        RETURN;
    END

    -- Delete show
    DELETE FROM [Show]
    WHERE show_Number = @SHNumber;

    PRINT 'Show removed';

END;
```

Description: This stored procedure deletes a scheduled show from the system. It ensures that the show exists and matches the correct hall and time before deletion, preventing accidental removal of wrong records.

1.2.20 SearchMovie (Stored Procedure)

```
CREATE PROCEDURE SearchMovie
    @Keyword VARCHAR(100)
AS
BEGIN
    SELECT *
    FROM Movie
    WHERE Title LIKE '%' + @Keyword + '%'
        OR EIDR LIKE '%' + @Keyword + '%';
END;
```

Description: This stored procedure is used to search for movies based on a keyword. It allows users to find movies either by title or by EIDR using partial matching. This improves search flexibility and user experience in the system.

1.2.21 SP_Schedule_show (Stored Procedure)

```
CREATE PROCEDURE SP_Schedule_show
    @Hall_num INT,
    @time DATETIME,
    @adminID INT,
    @price DECIMAL(10,2),
    @EIDR VARCHAR(20)
AS
BEGIN

    -- Check if show number already exists
    IF EXISTS (SELECT 1 FROM [Show] WHERE show_Number = @SHNumber)
    BEGIN
        PRINT 'ERROR: Show Number already exists.'
        RETURN;
    END

    -- Check if hall is already booked at same time
    IF EXISTS (
        SELECT 1
        FROM [Show]
        WHERE HallNumber = @Hall_num
        AND Showtime = @time
    )
    BEGIN
        PRINT 'This time in this hall is fully booked'
        RETURN;
    END
END
```

```

-- Insert new show
INSERT INTO [Show]
(show_Number, Price, Showtime, show_admin_id, sh_EIDR, HallNumber)
VALUES
(@SHNumber, @price, @time, @adminID, @EIDR, @Hall_num);

PRINT 'Scheduling Successful';

END;

```

Description: This stored procedure schedules a new movie show in the system. It ensures that each show number is unique and that no hall is double-booked at the same time. If all conditions are valid, it inserts a new show linked to a movie, hall, and admin.

1.2.22 Delete Movie (Stored Procedure)

Create PROCEDURE [dbo].[DeleteMovie]

```

@MovieEIDR VARCHAR(20)

AS

BEGIN

SET NOCOUNT ON;

BEGIN TRY

if NOT EXISTS (

    SELECT 1

    FROM Movie

    WHERE EIDR = @MovieEIDR

)

BEGIN

    select 'Movie with EIDR ' + @MovieEIDR + ' does not exist' as Result;

    RETURN;

END

DELETE FROM Movie

WHERE EIDR = @MovieEIDR;

select 'success ' as Result;

```



```
END TRY
BEGIN CATCH
    select 'error ' as Result;
END CATCH
END;
```

1.2.23 GetAllMoviesWithGenre (Stored Procedure)

```
CREATE PROCEDURE GetAllMoviesWithGenre
    @Genre VARCHAR(50)
AS
BEGIN
    SELECT
        m.EIDR,
        m.Title,
        m.Duration,
        m.AgeRating
    FROM Movie m
    LEFT JOIN Movie_Genre mg
    ON m.EIDR = mg.EIDR
END;
```

1.3 Queries

1.3.1 Seat Availability

```
SELECT s.HNumber, COUNT(*) AS AvailableSeats
FROM Seat s
LEFT JOIN Ticket t
    ON s.HNumber = t.HNumber
    AND s.SeatNumber = t.SeatNumber
WHERE t.SeatNumber IS NULL
GROUP BY s.HNumber;
```

Description: Finds all seats that have not yet been assigned a ticket by using a LEFT JOIN between Seat and Ticket, then filtering for rows where no match was found (SeatNumber IS NULL). Groups results by hall to show the count of available seats per hall.

1.3.2 Movies by Genre

Query 1: All Movies with Their Genre

```
SELECT m.EIDR, m.Title, m.Duration, m.AgeRating, mg.Genre
FROM Movie m, Movie_Genre mg
WHERE m.EIDR = mg.EIDR;
```

Description: Joins Movie and Movie_Genre to display each movie alongside its genre(s). A movie with multiple genres appears on multiple rows.

Query 2: Count of Movies per Genre

```
SELECT Genre, COUNT(*) AS NumberOfMovies
FROM Movie_Genre
GROUP BY Genre;
```

Description: Groups all genre entries and counts how many movies belong to each genre, giving an overview of the genre distribution across the catalog.

Query 3: Movie Titles with Genres (Sorted)

```
sql
SELECT m.Title, mg.Genre
FROM Movie m, Movie_Genre mg
```

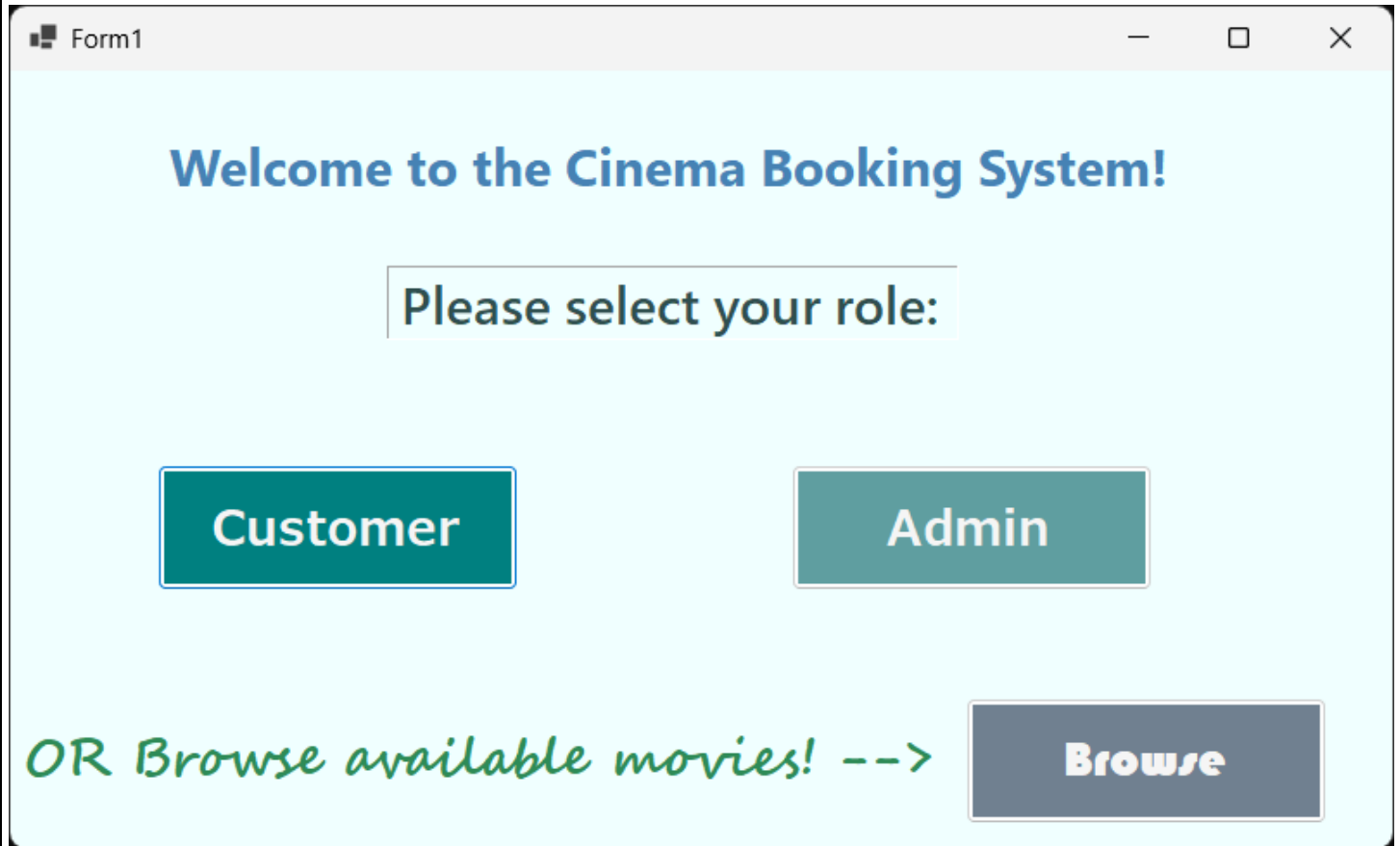
WHERE m.EIDR = mg.EIDR

ORDER BY m.Title;

Description: A simplified view listing each movie title next to its genre, ordered alphabetically by title for readability.

2. GUI and Database Interface:

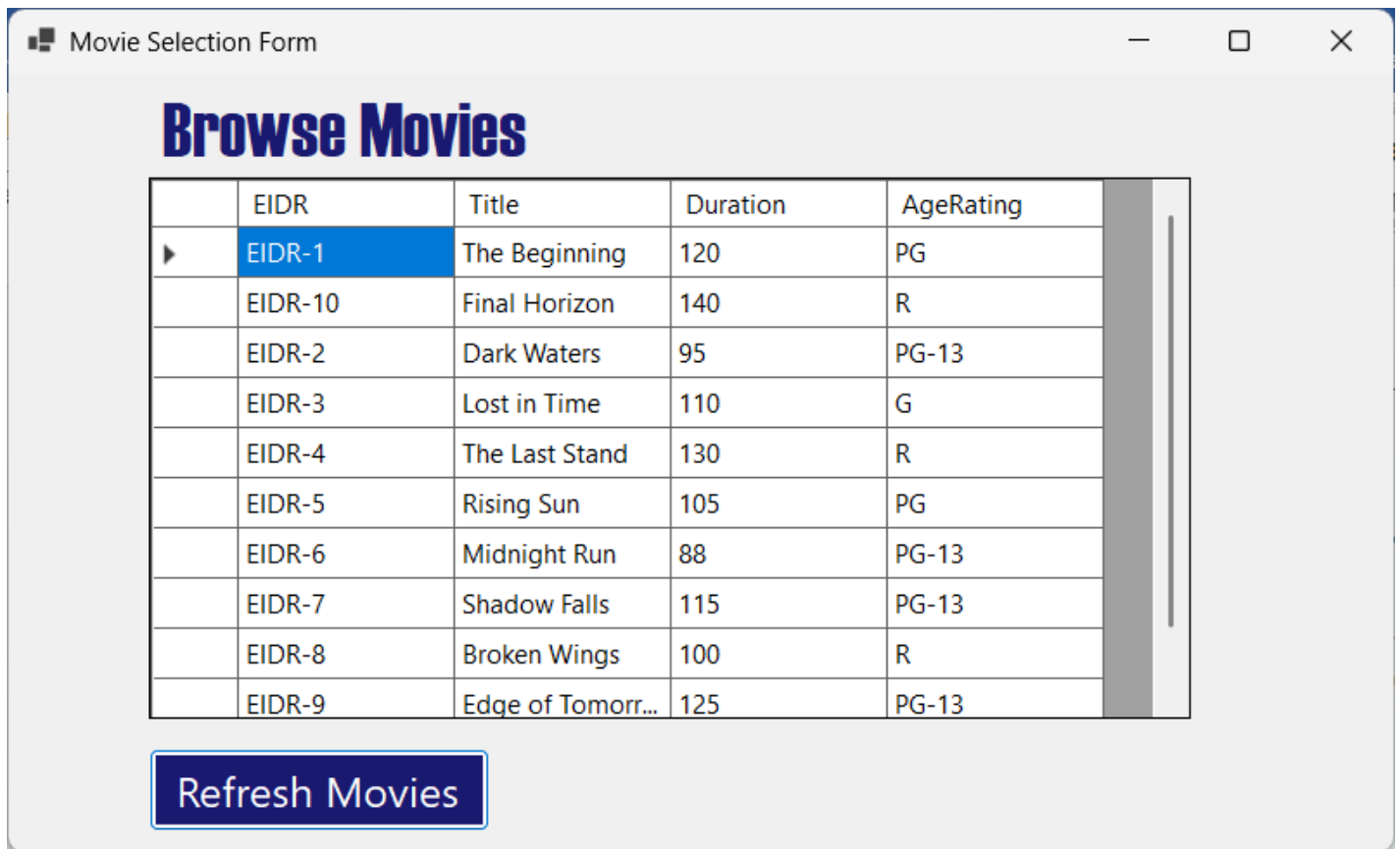
2.1 User Form



The screenshot shows a window titled "Form1" with a light blue background. At the top, it says "Welcome to the Cinema Booking System!". Below this, there is a text box containing "Please select your role:". Underneath the text box are two buttons: "Customer" (dark teal) and "Admin" (medium teal). At the bottom left, there is a green cursive text "OR Browse available movies! -->". To the right of this text is a "Browse" button (dark grey).

The first thing that we see once, the program is run is the [User Form](#). From there the user chooses his position: either a customer or an administrator. Or you could be a guest and just check out available movies.

2.2 Movie Selection Form



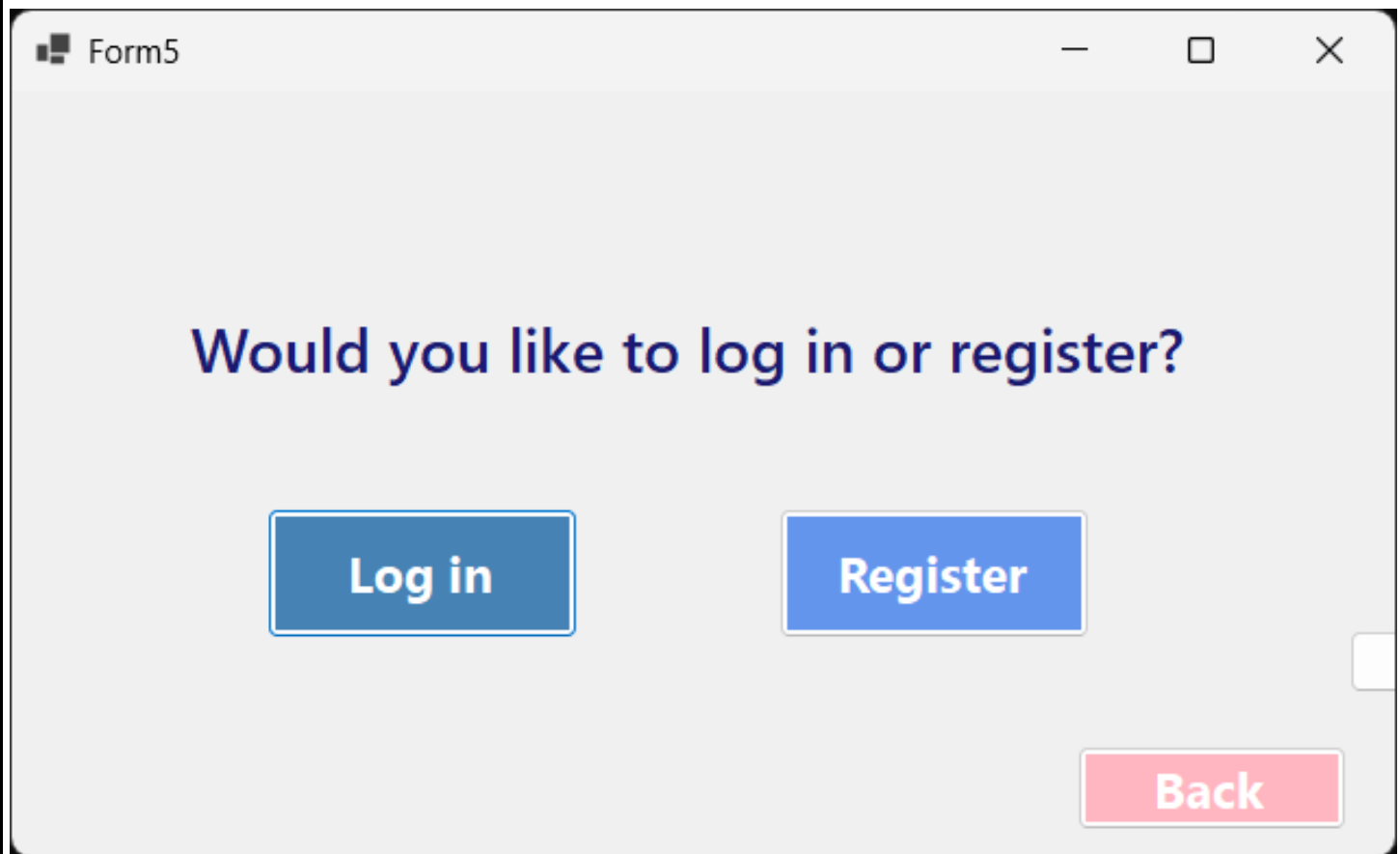
	EIDR	Title	Duration	AgeRating
▶	EIDR-1	The Beginning	120	PG
	EIDR-10	Final Horizon	140	R
	EIDR-2	Dark Waters	95	PG-13
	EIDR-3	Lost in Time	110	G
	EIDR-4	The Last Stand	130	R
	EIDR-5	Rising Sun	105	PG
	EIDR-6	Midnight Run	88	PG-13
	EIDR-7	Shadow Falls	115	PG-13
	EIDR-8	Broken Wings	100	R
	EIDR-9	Edge of Tomorr...	125	PG-13

Refresh Movies

You are redirected to this form if you clicked on the **Browse** button. This form uses `string sql = "SELECT EIDR, Title, Duration, AgeRating FROM Movie";` from the SQL **Movie Table**; which selects statements to display available movies in the database.

2.3 Customer Interface

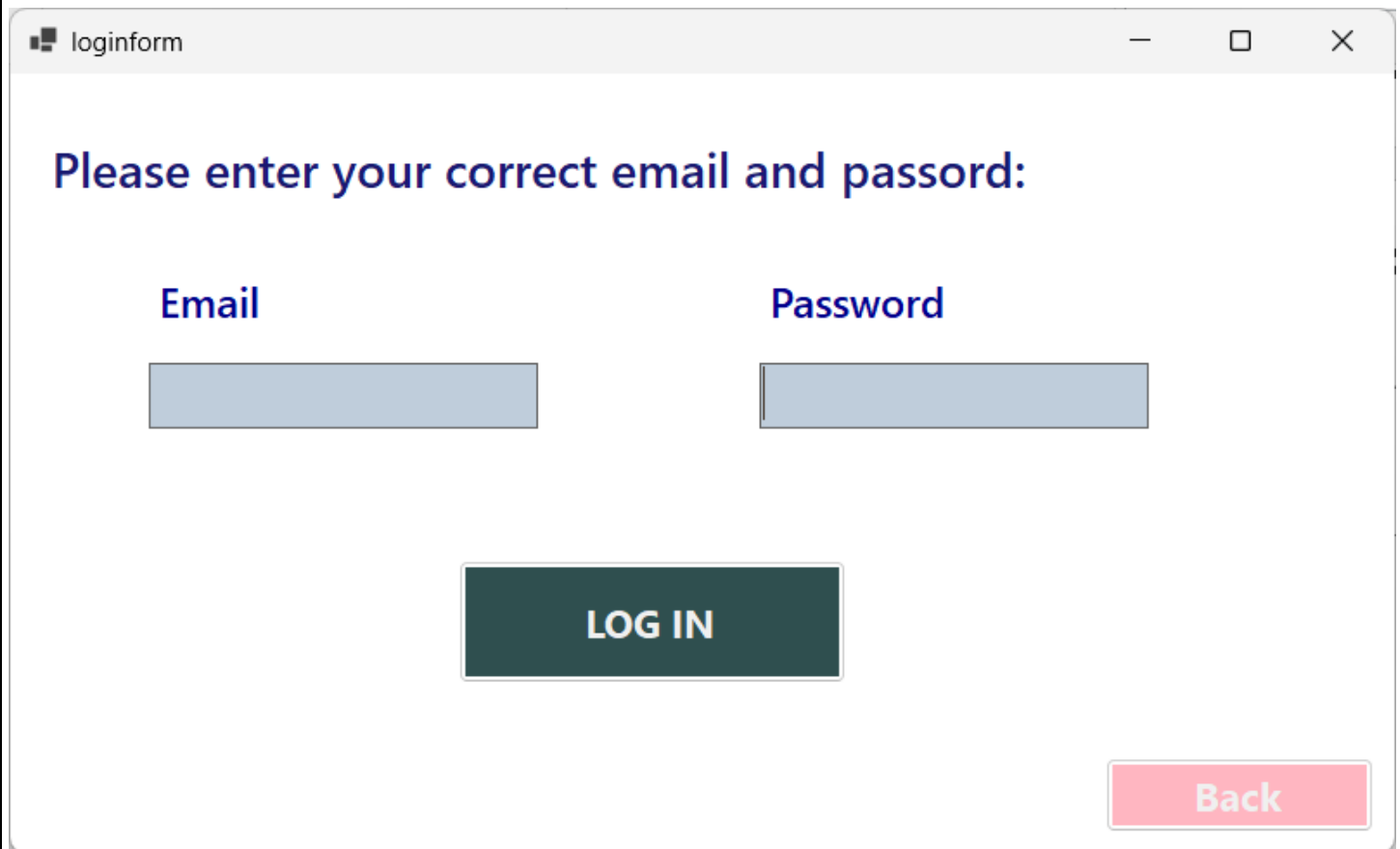
2.3.1 Customer Options



The screenshot shows a web browser window titled "Form5". The main content area has a light gray background and contains the text "Would you like to log in or register?" in a dark blue font. Below this text are three buttons: a blue "Log in" button, a blue "Register" button, and a pink "Back" button located at the bottom right. The browser window also shows standard window controls (minimize, maximize, close) in the top right corner.

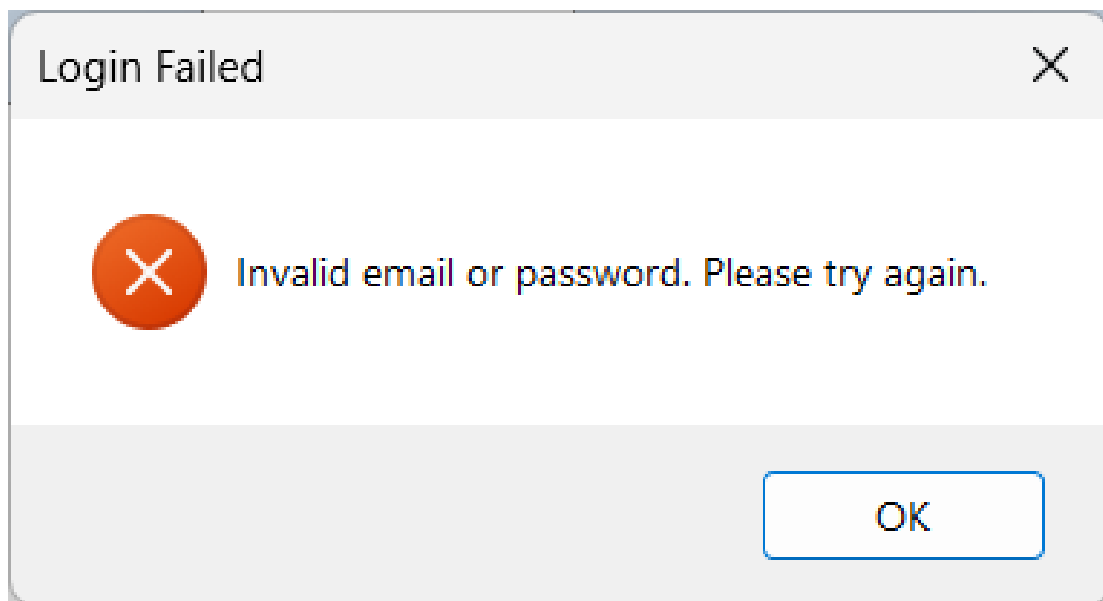
Once you click on the **Customer** button you will be redirected to a page where you will choose between logging in (if you already have an account) or registering (if it's your first time using the site) so we can add a new customer with new details.

2.3.2 Log in Form



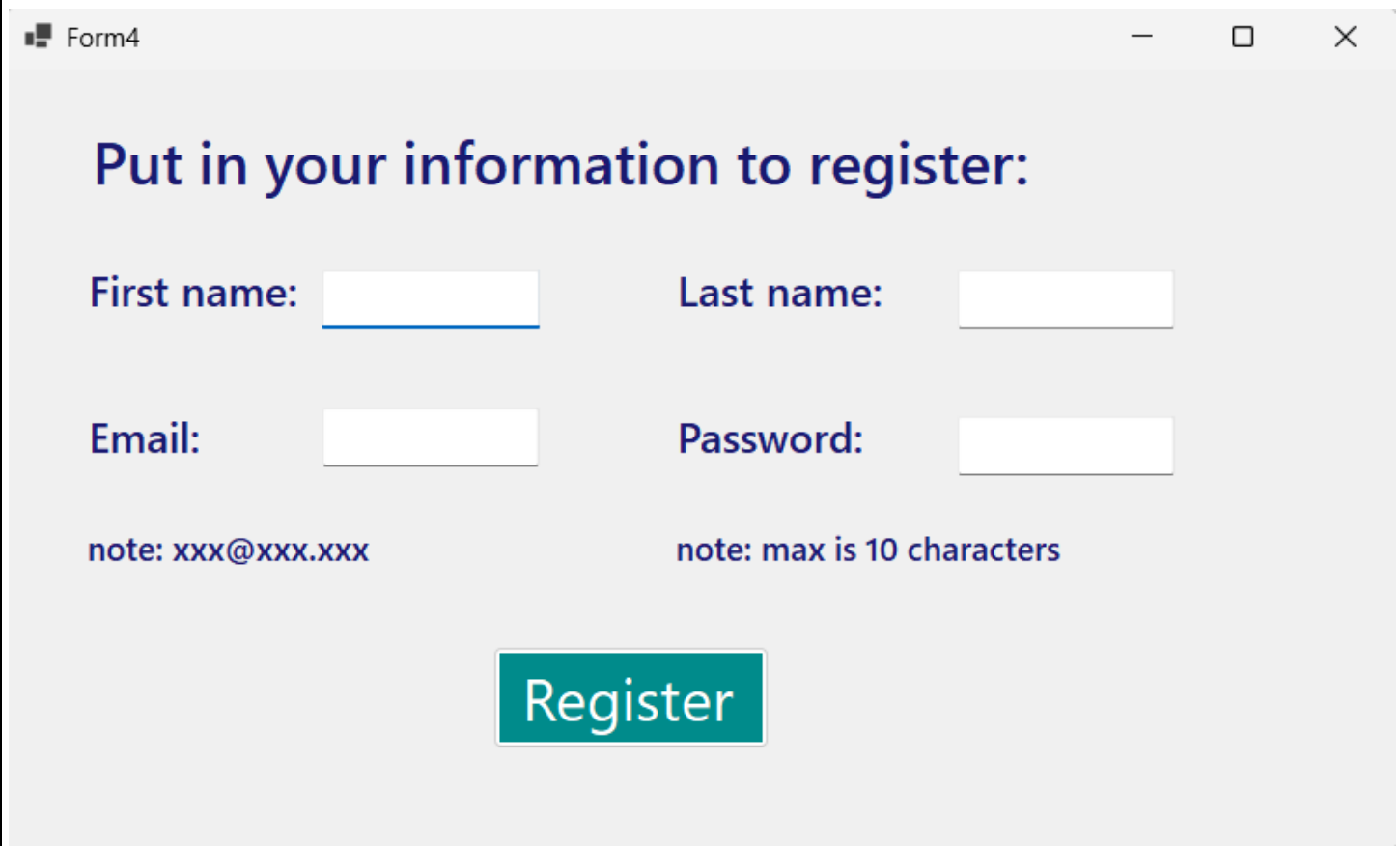
The screenshot shows a web browser window titled "loginform". Inside the window, there is a heading "Please enter your correct email and passord:". Below this heading, there are two input fields: "Email" and "Password". The "Email" field is a light blue rectangle, and the "Password" field is a light blue rectangle with a small vertical line on its left side. Below these fields is a dark green button with the text "LOG IN" in white. In the bottom right corner of the window, there is a pink button with the text "Back" in white.

To log in, the user must add his email and password then click on the **Log In** button. However, if it's a new customer and its information isn't in the database and not registered; a **Login Failed** error message will be displayed.



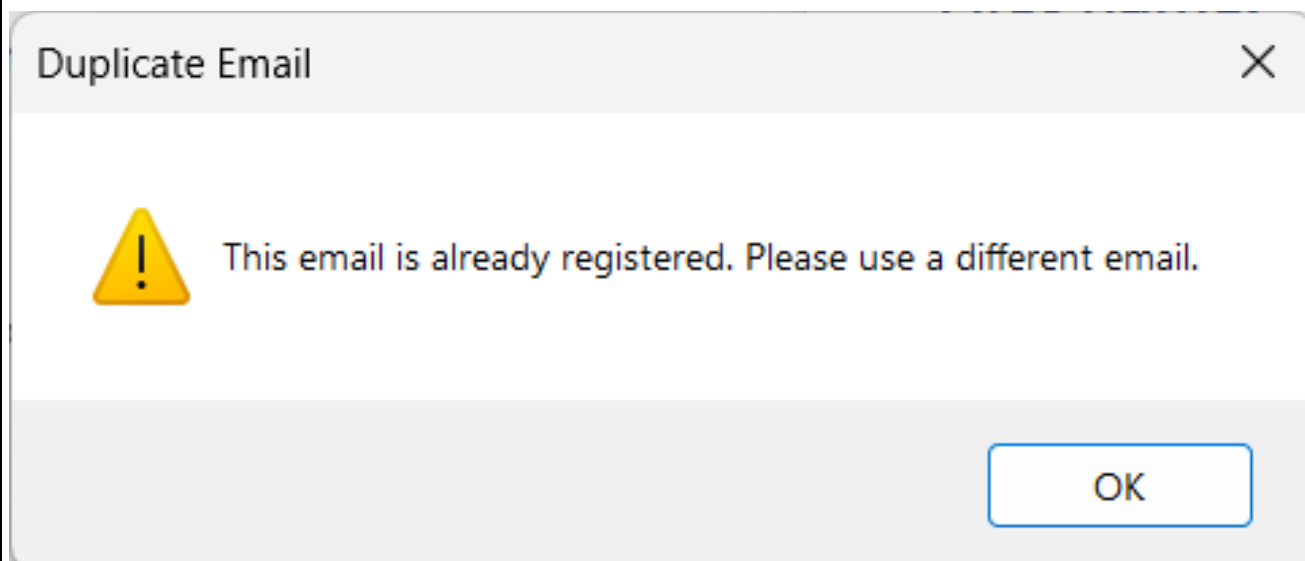
But if user has an account and has entered his correct information Logging in is done using procedure **CustomerLoginWithDetails** from the Stored Procedures & Functions' Query.

2.3.3 Register Form



The screenshot shows a window titled "Form4" with a light gray background. At the top, there are standard window controls (minimize, maximize, close). The main heading is "Put in your information to register:" in a large, dark blue font. Below this, there are four input fields arranged in a 2x2 grid. The first row contains "First name:" and "Last name:", each followed by a white text box. The second row contains "Email:" and "Password:", each followed by a white text box. Below the "Email:" field, there is a note: "note: xxx@xxx.xxx". Below the "Password:" field, there is a note: "note: max is 10 characters". At the bottom center, there is a green rectangular button with the word "Register" in white text.

Once the user clicks on the **Register** button, they are redirected to the Register Form; where the users are required to fill in their first and last name as well as their email and password following constraints and they click on the **Register** button to go to the next step. However, in case of previously saved email in the database, a **Duplicate Email** warning message is displayed. The user cannot register with the registered email, and they must add a new email address. Or an **Invalid Email** warning message is displayed.



The screenshot shows a dialog box titled "Duplicate Email" with a close button (X) in the top right corner. Inside the dialog, there is a yellow warning triangle icon with an exclamation mark. To the right of the icon, the text reads: "This email is already registered. Please use a different email." At the bottom right of the dialog, there is a white rectangular button with a blue border and the text "OK".

Invalid Email



Invalid email format. Please enter a valid email (e.g. name@example.com).

OK

The registration is done using the [AddCustomerFinal](#) procedure from Stored Procedures & Functions' Query.

Form4



Put in your information to register:

First name:

Last name:

Email:

Password:

note: xxx@xxx.xxx

note: max is 10 characters

Register

In case of a successful registration, the form is automatically closed, and the now customer is back to the Welcome Dashboard; where they click on the [Log in](#) button and proceed to fill in their new customer details.

2.3.4 Customer Dashboard Form

Welcome, Sara Mohamed

— □ ×

Welcome back, Sara Mohamed!

Back

Manage Bookings

Add phone

refresh grids

Delete phone

DELETE ACCOUNT

	Customer_	FirstName	LastName	Email	Booking_II	Booking_d
▶	2	Sara	Mohamed	sara@g...	2	4/7/2026...
•						

The customer is met with the [Dashboard and Welcome Form](#), where they can see their saved details and information at the bottom of the form. They can also manage their Booking, add or delete their phone number, refresh the page in case they add any new information, or they can delete their account altogether.

In this form, the customer can:

1. [Manage Bookings](#) → redirects the customer to the [Booking Page](#) (explained in detail in Page 39).
2. [Phone grid](#) → shows the saved phone numbers in the system. This operation is done using the [GetCustomerPhones](#) procedure from Stored Procedures & Functions' Query.
[Add Phone](#) → leads us to a small form where the customer can add a new phone number. This

operation is done using the [AddCustomerPhone](#) procedure.

3. [Refresh grids](#) → refreshes both phone grids (at the top left of the page) and booking to make sure it's up to date
4. [Delete Phone](#) → the customer selects the number they want to delete then click [Delete phone](#). This operation is done using the [DeleteCustomerPhone](#) procedure from Stored Procedures & Functions' Query.

Customer	Booking_Id	Booking_S	Booking_d	B_shownur	Payment_Id
2	2	Confirm...	4/7/2026...	2	2

5. [Delete account](#) → This operation is done using the [DeleteCustomer](#) procedure from Stored Procedures & Functions' Query.

2.3.5 Booking Form

Cinema Booking System - Seat Selection

Booking Page

Create New Booking

Movie EIDR:

Load

Show Time:

Number of Tickets:

Hall: -

Click to select seats (select needed):

Available Seats:

Please choose as many seats as your number of tickets

Selected Seats:

Proceed to Payment

BROWSE MOVIES

View Ticket Details

Booking ID:

View All Ticket Details

BACK

Manage Existing Bookings

	Booking_ID	Booking_Status	Booking_date	B_shownumber	Customer_ID
▶	2	Confirmed	4/7/2026 11:30 ...	2	2
•					

Target Booking ID:

Cancel

Check Status

Status will appear here

New Method:

Update Method

Once the customer clicks on the [Manage Bookings](#) button in the [Customer Dashboard](#) form, this [Booking Page](#) is loaded. The customers can search and browse for movies, pick a showtime, select their seats, pay, and manage their bookings (view, cancel, update payment method, check status).

The customer can click on:

1. **Movie EIDR** → loads all available movies' unique identifiers (the customers have them from when they were movie browsing).
2. **Show Time** → loads all available timeslots of the chosen movie.
3. **Number of Ticket** → always set to 1 by default to avoid Ticket ID repeating and not being unique.
4. **Available Seats** → loads all available seats, where all seats are tracked.
5. **Booking ID** → automatically generated.
6. **Proceed to Payment** → open the payment form, where the customer views the total amount, chooses the payment method and click on **Confirm & Save**.

Cinema Booking System - Seat Selection

Booking Page

Create New Booking

Movie EIDR: **Load**

Show Time:

Number of Tickets:

Hall: **3**
Click to select seats (2 needed):

Available Seats:

Please choose as many seats as your number of tickets

Selected Seats:

Proceed to Payment

Checkout

Complete Payment

Payment Method:

Total Amount:

 **Confirm & Save**

BACK

7. **View All Tickets Details** → load the **Booking Tickets Details**.

BOOKING TICKETS DETAILS (2)

Movie: Lost in Time
Show Time: 4/7/2026 12:45:00 PM
Date: 2026-05-03
Hall: 3
Booking Status: Confirmed
Booking ID: 1027

TICKET #1

Ticket ID: 1014
Status: Booked
Seat: 1 (Front-Right)

TICKET #2

Ticket ID: 1015
Status: Booked
Seat: 2 (Front-Left)

Total Tickets: 2

BROWSE MOVIES

View Ticket Details

Booking ID:

View All Ticket Details

BACK

date	B_shownumber	Customer_ID
3:24 P...	1	13
3:26 P...	1	13
3:26 P...	1	13
1:34 A...	3	13

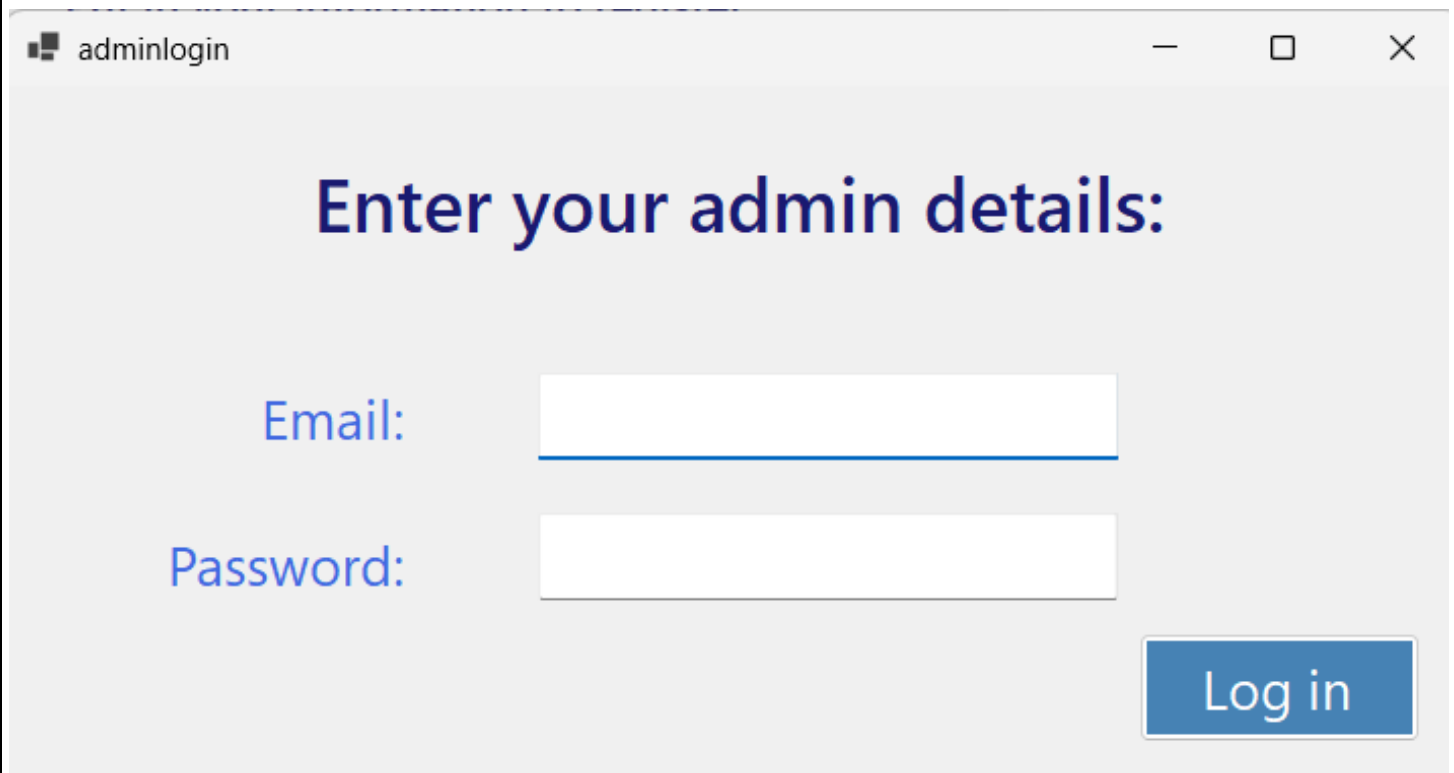
Booking Page Flow Summary

1. Customer enters an EIDR (movie ID) → showtimes load into dropdown
2. Customer picks a showtime → hall number is set
3. Customer sets ticket count → available seats load
4. Customer picks seats one by one → seats are tracked
5. Customer clicks "Proceed to Payment" → payment form opens → tickets inserted → booking confirmed

Note that all procedures used for these operations are mentioned in the report's Appendix.

2.4 Admin Interface

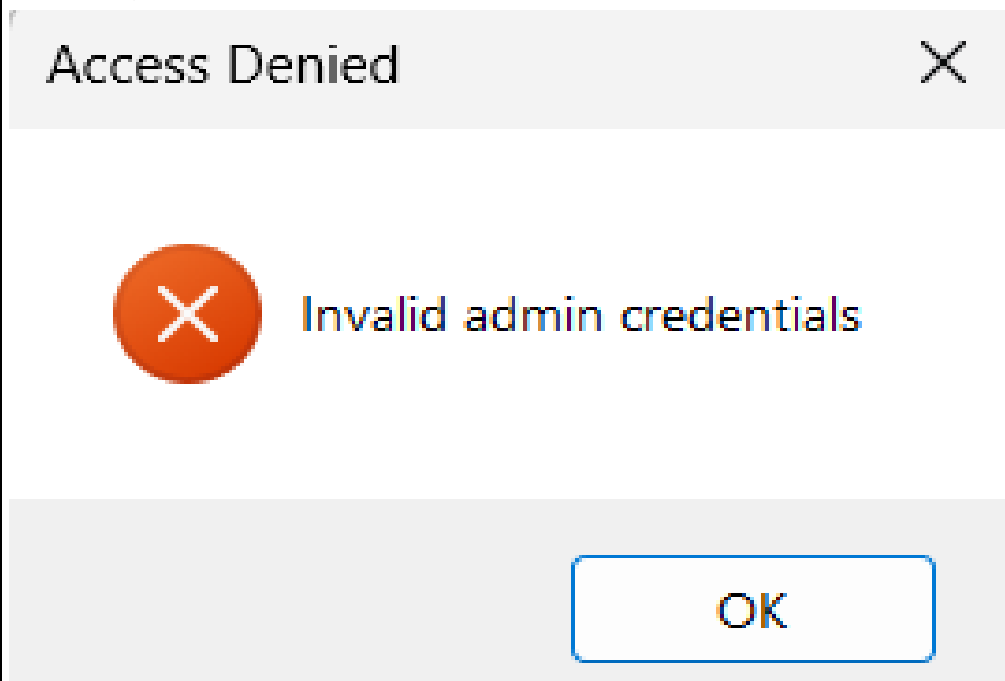
2.4.1 Admin Log in Form



The image shows a web browser window titled "adminlogin". Inside the window, there is a heading "Enter your admin details:". Below this heading, there are two input fields: "Email:" and "Password:". To the right of the "Password:" field is a blue button labeled "Log in".

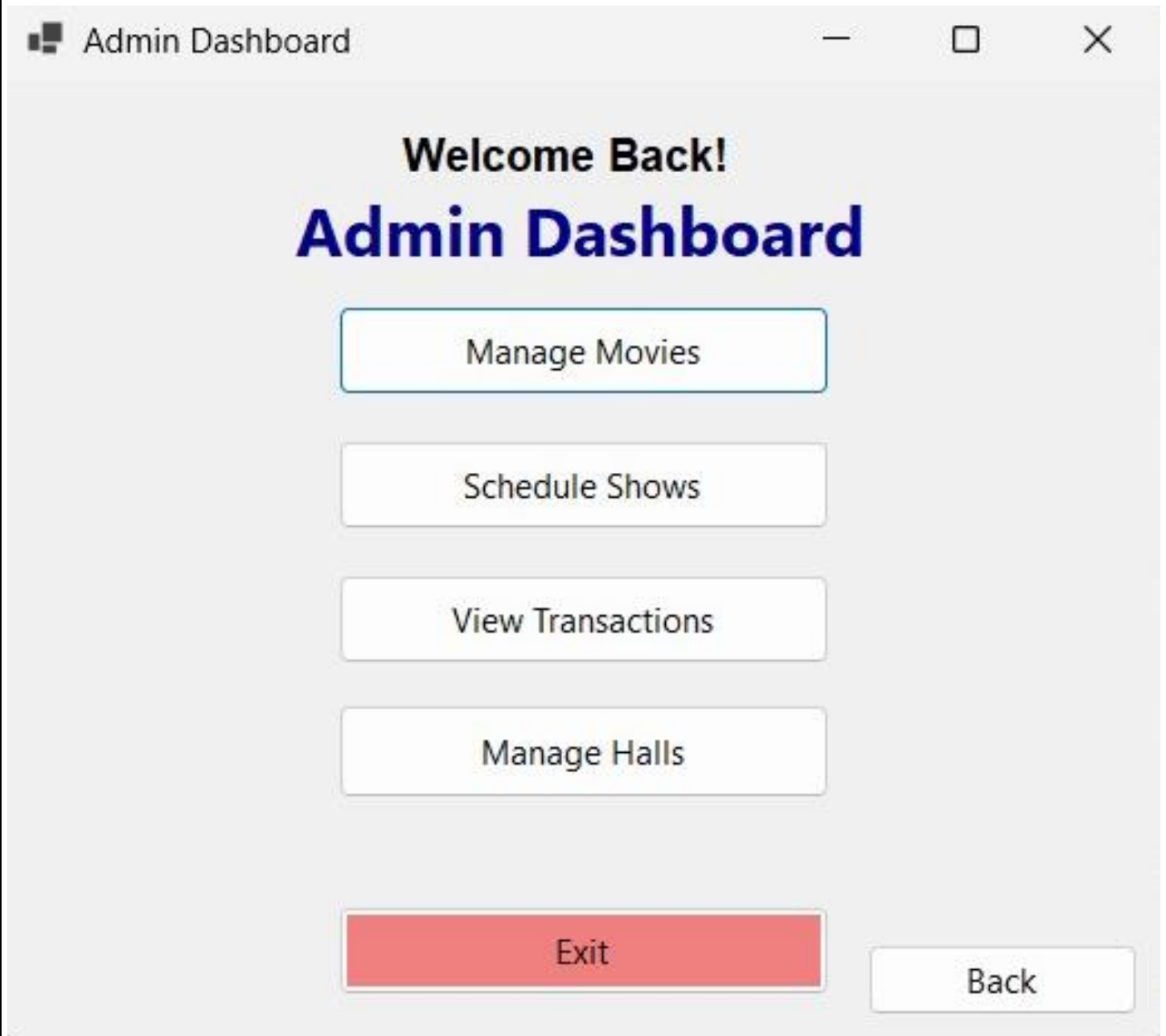
If the user were to click on the **Admin** button when choosing from the **User Form**, the user will now have to fill in their admin details and click on **Log in** button to be able to access the **Admin Dashboard**. This form uses `string query = "SELECT COUNT(*) FROM admin WHERE admin_email = @admin_email AND admin_password = @admin_password";` to be able to access the **Admin Table** in the Database's query.

However, in case of invalid details or admin information an **Access Denied** error message is displayed.



The image shows a dialog box with a light gray background. At the top, it says "Access Denied" in a large, bold, black font. To the right of this text is a black "X" icon. Below this, there is a red circular icon with a white "X" inside. To the right of this icon, the text "Invalid admin credentials" is displayed in a blue, bold font. At the bottom of the dialog box, there is a white button with a blue border labeled "OK".

2.4.2 Admin Dashboard



Once the admin logs in successfully, he can control and supervise the booking system by clicking on:

1. [Manage Movies](#) → redirects the admin to the [Movie Management](#) Form (detailed in Page 43).
2. [Schedules Shows](#) → redirects the admin to the [Scheduling Dashboard](#) Form (detailed in Page 45).
3. [View Transactions](#) → redirects the admin to the [Financial Management](#) Form (detailed in Page 48).
4. [Manage Halls](#) → redirects the admin to the [Hall Management](#) Form (detailed in Page 49).

2.4.3 Movie Management Dashboard

EIDR	Title	Duration	AgeRating	Genre
EIDR-1	The Beginning	120	PG	Action
EIDR-1	The Beginning	120	PG	Sci-Fi
EIDR-10	Final Horizon	140	R	
EIDR-2	Dark Waters	95	PG-13	Action
EIDR-2	Dark Waters	95	PG-13	Thriller
EIDR-3	Lost in Time	110	G	Drama
EIDR-3	Lost in Time	110	G	Romance
EIDR-4	The Last Stand	130	R	Animation
EIDR-4	The Last Stand	130	R	Comedy
EIDR-5	Rising Sun	105	PG	Horror
EIDR-6	Midnight Run	88	PG-13	Adventure

The admin can view the movie details (like its Title, Duration, Age Rating, and Genre) in the **Movie Grid** at the right of the dashboard. The admin can also click on:

1. **Movie Grid** → loads all the available movies at the right of the dashboard.
2. **Add Movie** → adds new movies to the **Movie Grid** and the **Movie Browser**, after the admin fills in the **Movie EIDR**, its **title**, **duration**, **age** for age rating, **genre**.
3. **Delete Movies** → deletes a selected movie and remove it from **the Movie Grid**.
4. **Search Movie** → searches for the movie in the **Movie Grid**.
5. **Search with Genre** → searches for the movie in the **Movie Grid** by genre.
6. **Add Genre** → adds a new genre of movies.
7. **Reset** → resets the system.

This dashboard operates using the following procedure:

1. **GetAllMoviesWithGenres** → to load available movies and reset by clicking reset
2. **AddGenreToMovie** → to add a movie genre by entering its details and THEN clicking add genre
3. **AddMovie** → to add a new movie by entering all details then clicking add movie, cannot add a movie already available and have to enter details in specific format
4. **DeleteMovie** → just need to add EIDR to delete movie then click delete
5. **SearchMovie** → by entering movie title or EIDR then click search movie which will be shown on grid
6. **GetMoviesByGenre** → enter genre you want to search for and filter will be applied to displayed grid

Movie Managment

EIDR

Title

Duration

Age

Genre

Search Title

	Title	AgeRating
▶	The Beginning	PG
	Final Horizon	R
	Dark Waters	PG-13
*		

Add Movie

Delete Movie

Search Movie

Movie Managment

Back

EIDR

EIDR-10

Title

Duration

Age

Genre

Search Title

	EIDR	Title	Duration	AgeRating	Genre
▶	EIDR-1	The Beginning	120	PG	Action
	EIDR-1	The Beginning	120	PG	Sci-Fi
	EIDR-10	Final Horizon	140	R	Action
	EIDR-2	Dark Waters	95	PG-13	Action
	EIDR-2	Dark Waters	95	PG-13	Thriller
	EIDR-3	Lost in Time	110	G	Drama
	EIDR-3	Lost in Time	110	G	Romance
	EIDR-4	The Last Stand	130	R	Animation
	EIDR-4	The Last Stand	130		
	EIDR-5	Rising Sun	105		
	EIDR-6	Midnight Run	88		

Add Movie

Delete Movie

Search Movie

Se

OK

Reset

Genre added successfully!

2.4.4 Scheduling Dashboard

The screenshot shows a web browser window titled "SchedulingForm". The main heading is "Scheduling Dashboard" in a large, bold, blue font. On the left side, there is a vertical stack of input fields with labels: "Hall Number", "Price", "AdminId", "Show Number", "Time", and "Show EIDR". Each label is in bold black text above its corresponding white input box. To the right of these fields is a large, empty gray rectangular area representing the "Show Grid". At the bottom of the dashboard, there are four buttons: "Schedule" (green), "Delete" (red), "View Shows" (blue), and "Back" (pink).

In the [Scheduling Dashboard](#), the admin can:

1. [Schedule](#) → schedule and add new shows by adding the [Hall Number](#), [Price](#), [AdminId](#), [Show Number](#), [Time](#) for the available timeslots, and [Show EIRD](#).
2. [View Shows](#) → displays all the scheduled shows on the [Show Grid](#) at the right of the dashboard.
3. [Delete](#) → deletes old shows by selecting the show and clicking on the [Delete](#) button or just go
4. [Back](#) → go back to the [Admin Dashboard](#).

Scheduling Dashboard

Hall Number

Price

AdminId

Show Number

Time

Show EIDR

	show_Number	Price	Showtime	show_admin_id	sh_EIDR	HallNur
▶	1	200.00	4/7/2026 10:00 ...	1	EIDR-1	1
	2	300.00	4/7/2026 11:30 ...	2	EIDR-2	2
	3	400.00	4/7/2026 12:45 ...	2	EIDR-3	3
	4	400.00	4/8/2026 9:15 A...	1	EIDR-4	4
	5	200.00	4/8/2026 2:00 P...	3	EIDR-5	5
	6	200.00	4/9/2026 10:30 ...	2	EIDR-6	6
	7	300.00	4/9/2026 12:30 ...	3	EIDR-7	7
	8	400.00	4/9/2026 4:30 P...	2	EIDR-8	8
	9	200.00	4/10/2026 9:15 ...	1	EIDR-9	9
	10	200.00	4/10/2026 11:1 ...	2	EIDR-10	10

Schedule

Delete

View Shows

Back

This dashboard operates using the following procedure:

Clicking on view shows-> loads available shows using "SELECT * FROM [Show]"

Procedure DELETE_Scheduled_show-> by entering details first then clicking on delete,

deletes the show details SP_Schedule_show-> by entering details then clicking schedule to add new show details

However, any invalid inputs

Scheduling Dashboard

Hall Number

Price

AdminId

Show Number

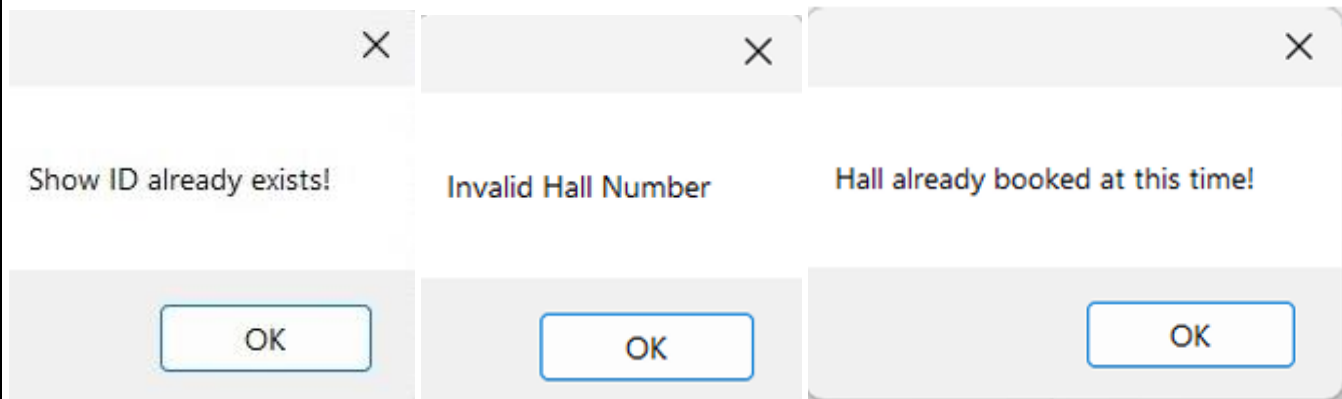
Time

Show EIDR

mber	Price	Showtime	show_admin_id	sh_EIDR	HallNumber
	400.00	4/7/2026 12:45 ...	2	EIDR-3	3
	400.00	4/8/2026 9:15 A...	1	EIDR-4	4
	200.00			EIDR-5	5
	200.00			EIDR-6	6
	300.00			EIDR-7	7
	400.00			EIDR-8	8
	200.00			EIDR-9	9
	300.00			EIDR-10	10
*					

Invalid input

OK



2.4.5 Transaction Interface

Admin Dashboard

Financial Managment

Today's Revenue: \$300.0000

	transaction_id	transaction_datet	transaction_type	transaction_status	payment_id
▶	1	3/1/2026 10:00 ...	payment	completed	1
	2	4/2/2026 2:30 P...	payment	pending	2
	3	1/3/2026 9:15 A...	refund	failed	3
	4	4/24/2026 3:51 ...	payment	completed	1
	5	5/2/2026 12:10 ...	payment	completed	11
	6	5/2/2026 6:43 A...	payment	completed	12
	7	5/2/2026 7:18 A...	payment	completed	13
	8	5/2/2026 7:30 A...	payment	completed	14
	9	5/2/2026 7:37 A...	payment	completed	15
	10	5/2/2026 7:38 A...	payment	completed	16
	11	5/2/2026 8:24 A...	payment	completed	17
	12	5/2/2026 8:27 A...	payment	completed	18
	13	5/2/2026 8:37 A...	payment	completed	19
	14	5/2/2026 8:38 A...	payment	completed	20
	15	5/2/2026 8:47 A...	payment	completed	21

Add Transaction

choose transaction type

choose transaction status

type payment ID

Filter Transaction by Status

choose transaction status

Load Transactions

Back

The admin can access the [Financial Management](#) dashboard once he successfully logged in. In this form, the admin can:

1. [Add Transaction](#) → requires the admin to enter the transaction type, transaction status, and payment ID to avoid any errors. This operation is done using the [log_new_transaction](#) procedure from the Stored Procedures & Functions' Query.
2. [Filter Transaction by Status](#) → requires the admin to enter the transaction status to avoid any errors. This operation is done using the [get_transaction_status](#) procedure from the Stored Procedures & Functions' Query.
3. [Load Transactions](#) → loads all transactions in the [Transaction Grid](#) at the left. This is done using the [SELECT * FROM transactions](#)
Today's revenue is done by -> "[SELECT dbo.fn_getDailyRevenue\(GETDATE\(\)\)](#)" from the [Transaction Table](#) in the Database.

2.4.6 Hall Management Dashboard

Form1

Back

Show Hall Capacity

Enter Hall Number:

Capacity Result:

Show Capacity

Add Hall

Enter Hall Number:

Enter hall capacity:

Choose Hall Type:

Add Hall

Delete Hall

Enter Hall Number:

Delete Hall

Hall Managment

	HallNumber	HallCapacity	HallType
▶	1	20	VIP
	2	25	Standard
	3	30	VIP
	4	35	Standard
	5	40	VIP
	6	45	Standard
	7	50	VIP
	8	30	Standard
	9	25	VIP
	10	20	Standard
*			

The admin can access the [Financial Management](#) dashboard once he successfully logged in. In this form, the admin can:

1. [Show Capacity](#) → permits the admin to the hall capacity.
2. [Add Hall](#) → permits the admin to add a hall once, he fills in the [Hall Number](#), [capacity](#) and [type](#).
3. [Delete Hall](#) → permits the admin to delete a specific hall.
4. [Hall Grid](#) → permits the admin to see the halls, and their capacity and type.

Back

Hall Managment

Show Hall Capacity

Enter Hall Number: 10

Capacity = 20

Show Capacity

Add Hall

Enter Hall Number: 11

Enter hall capacity: 30

Choose Hall Type: Stand ▾

Add Hall

Delete Hall

Enter Hall Number:

Delete Hall

	HallNumber	HallCapacity	HallType
▶	1	20	VIP
	2	25	Standard
	3	30	VIP
	4	35	Standard
	5	40	VIP
	6	45	Standard
	7	50	VIP
	8	30	Standard
	9	25	VIP
	10	20	Standard
	11	30	Standard
•			

The hall Capacity operates using the following procedure:

SELECT dbo.CheckHallCapacity(@HallNumber)" by clicking on show capacity button,
if hall is not found an error message will be displayed

Can add hall using "INSERT INTO Hall (HallNumber, HallCapacity, HallType) VALUES (@id, @cap,
@type)" And filling in details and if hall already exists and error will be displayed,
there are also constraints on the hall capacity.

Can delete hall using SELECT COUNT (*) FROM [Show] WHERE HallNumber = @id", will display an error if
hall is used in shows.

However, any invalid inputs

Delete Hall

Enter Hall Number: 12

Delete Hall



Hall does not exist!

OK

[Back](#)

Show Hall Capacity

Enter Hall Number:

Capacity = 20

[Show Capacity](#)

Add Hall

Enter Hall Number: Enter hall capacity: Choose Hall Type: **Stand** ▾[Add Hall](#)

Delete Hall

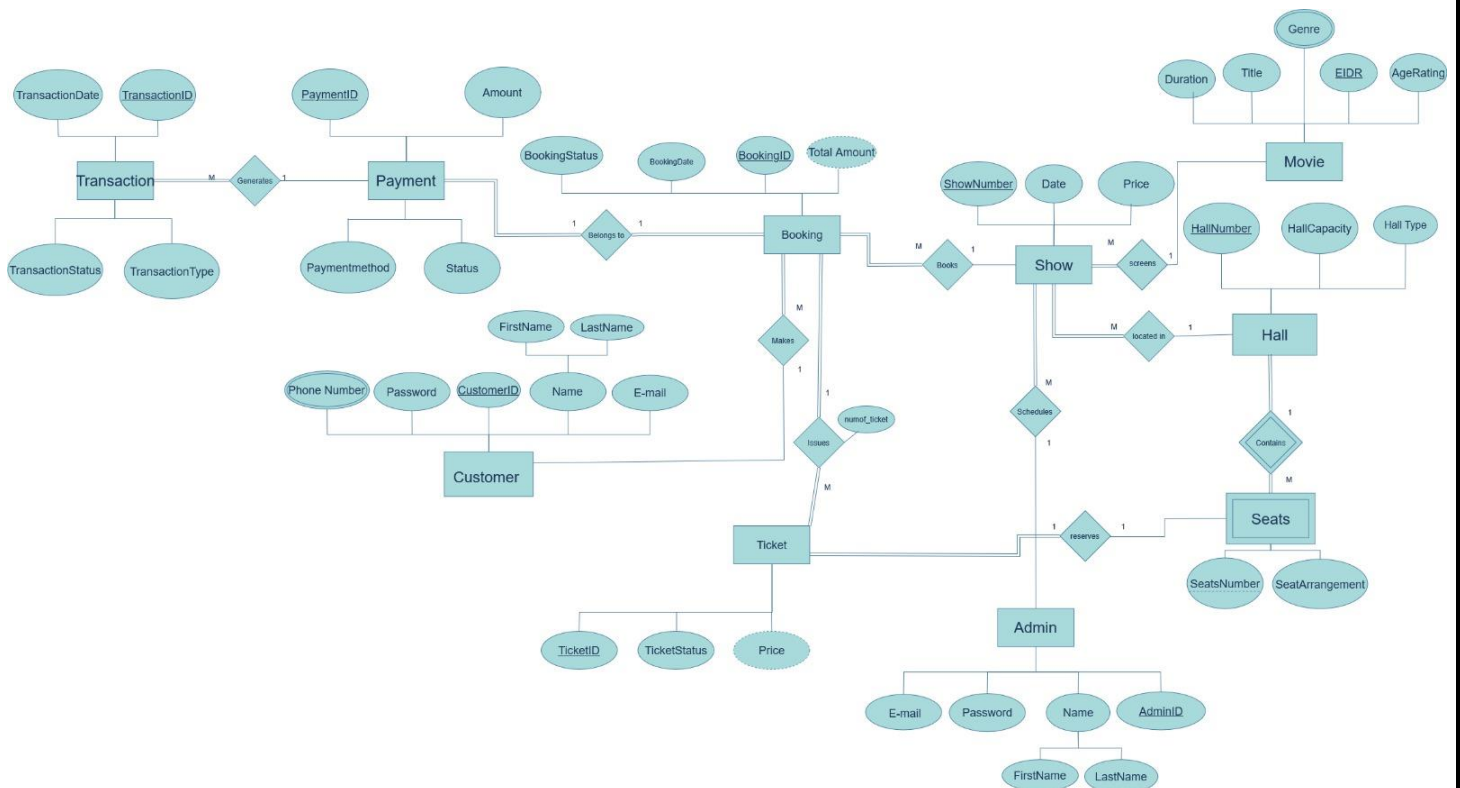
Enter Hall Number: [Delete Hall](#)

Hall Managment

	HallNumber	HallCapacity	HallType
▶	1	20	VIP
	2	25	Standard
	3	30	VIP
	4	35	Standard
	5	40	VIP
	6	45	Standard
	7	50	VIP
	8	30	Standard
	9	25	VIP
	10	20	Standard
*			

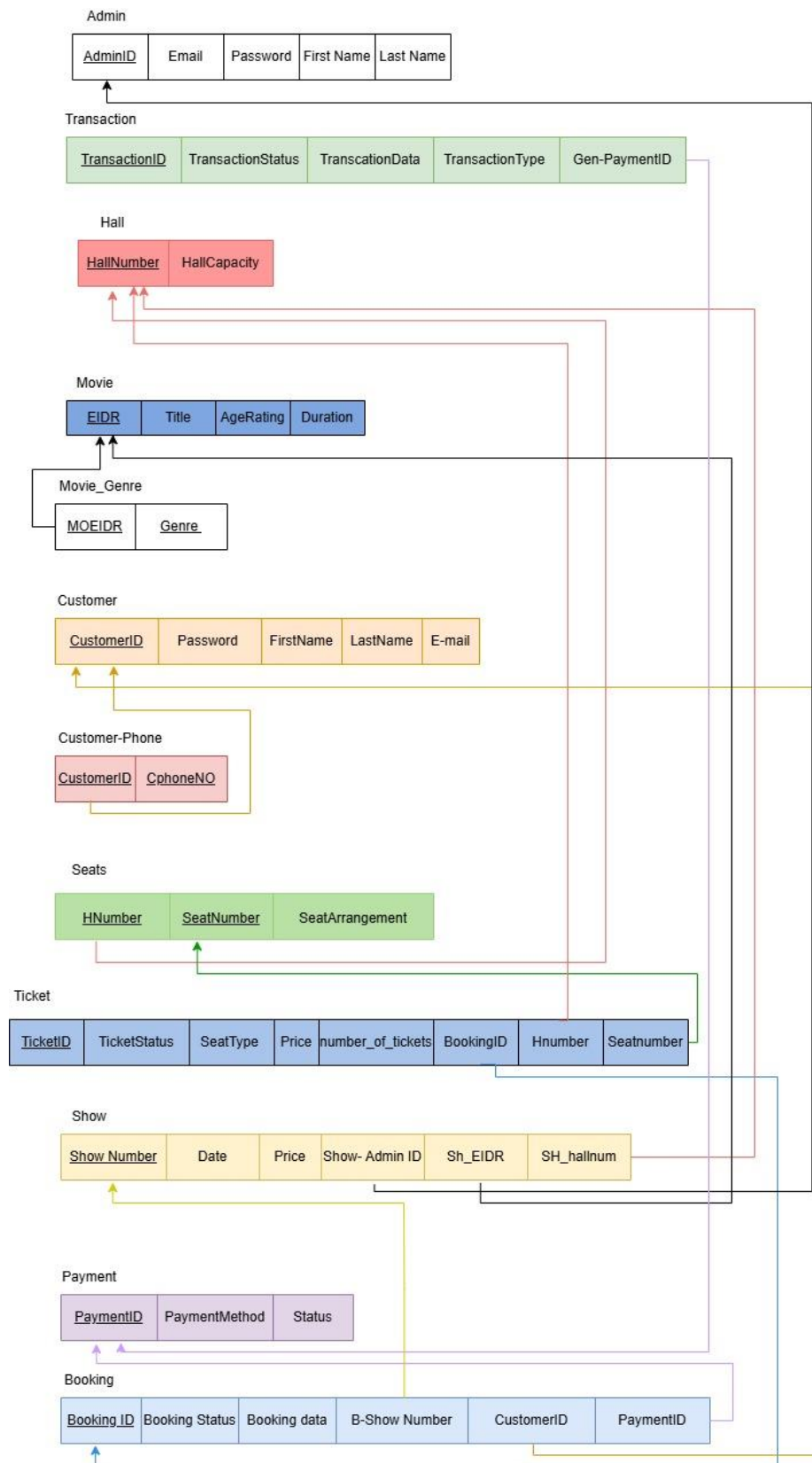
Appendix

ERD

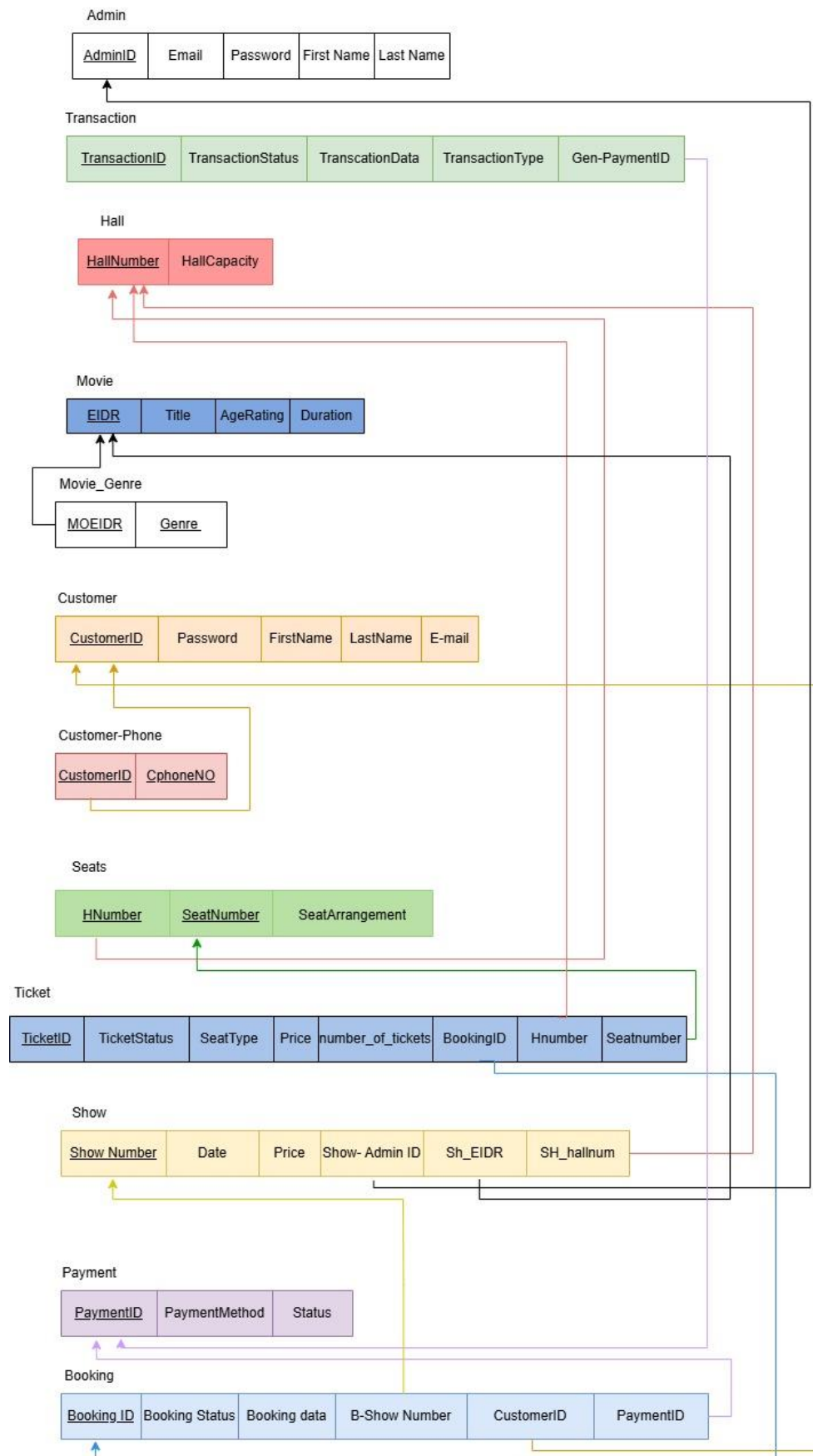


Schema

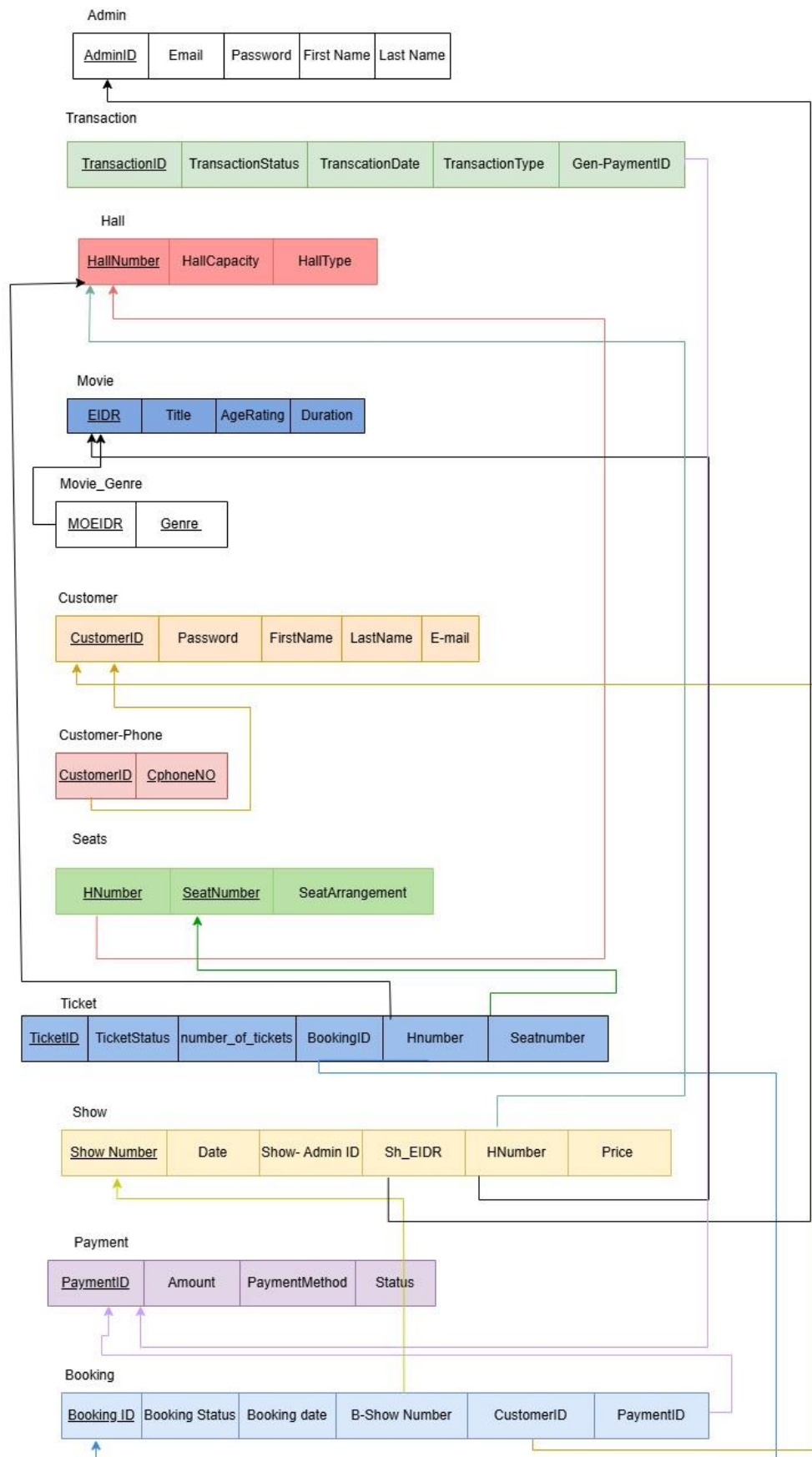
1 NF



2 NF



3 NF



All Queries Used in Booking Form

Location	Query	Purpose
LoadShowTimes	<pre>SELECT show_Number, Showtime, HallNumber, Price FROM Show WHERE sh_EIDR = @eidr</pre>	Gets all showtimes for the entered movie
LoadAvailableSeats	<pre>SELECT s.SeatNumber, s.SeatArrangement FROM Seat WHERE HNumber = @hall AND SeatNumber NOT IN (booked seats subquery)</pre>	Gets all seats in the hall that aren't already booked/pending
btnProceedToPayment	<pre>SELECT show_Number, Price FROM Show WHERE sh_EIDR = @eidr</pre>	Gets show number and price to calculate total amount
btnProceedToPayment	<pre>SELECT HallCapacity FROM Hall WHERE HallNumber = @hall</pre>	Checks hall capacity before booking
btnProceedToPayment	<pre>SELECT ISNULL(SUM(NumberOfTickets), 0) FROM Ticket JOIN Booking ...</pre>	Counts already-confirmed tickets to check remaining seats
btnViewTicket	Large JOIN across Ticket, Seat, Booking, Show, Movie	Retrieves full ticket details for a booking ID
btnStatus_Click	<pre>SELECT Booking_Status FROM Booking WHERE Booking_ID = @id</pre>	Returns current status of a booking
LoadBookings	<pre>SELECT * FROM Booking WHERE Customer_ID = @id</pre>	Loads all bookings for the current customer into the grid

1. Load Show Times

```
SELECT show_Number, Showtime, HallNumber, Price
FROM Show
WHERE sh_EIDR = @eidr
```

2. Load Available Seats

```
SELECT s.SeatNumber, s.SeatArrangement
FROM Seat s
WHERE s.HNumber = @hallNumber
AND s.SeatNumber NOT IN (
    SELECT DISTINCT t.SeatNumber
    FROM Ticket t
    JOIN Booking b ON t.BookingID = b.Booking_ID
    WHERE t.HNumber = @hallNumber
    AND b.B_shownumber = @showNumber
    AND b.Booking_Status IN ('Confirmed', 'Pending')
)
ORDER BY s.SeatNumber ASC
```

3. Get Show Info & Price (Proceed to Payment)

```
SELECT show_Number, Price
```

```
FROM Show
WHERE sh_EIDR = @eidr
```

4. Check Hall Capacity

```
SELECT HallCapacity
FROM Hall
WHERE HallNumber = @hall
```

5. Count Already Booked Tickets

```
SELECT ISNULL(SUM(t.NumberOfTickets), 0)
FROM Ticket t
JOIN Booking b ON t.BookingID = b.Booking_ID
WHERE t.HNumber = @hall
AND b.B_shownumber = @show
AND b.Booking_Status = 'Confirmed'
```

6. Create Booking

```
INSERT INTO Booking (Booking_Status, Booking_date, B_shownumber, Customer_ID)
VALUES ('Pending', GETDATE(), @show, @cust);
SELECT SCOPE_IDENTITY();
```

7. Insert Ticket Per Seat

```
INSERT INTO Ticket (TicketStatus, NumberOfTickets, BookingID, HNumber, SeatNumber)
VALUES ('Booked', 1, @bid, @hall, @seat)
```

8. Confirm Booking

```
UPDATE Booking
SET Booking_Status = 'Confirmed'
WHERE Booking_ID = @id
```

9. View Ticket Details

```
SELECT
    t.TicketID,
    t.TicketStatus,
    t.SeatNumber,
    t.HNumber,
    s.SeatArrangement,
    m.Title,
    sh.Showtime,
    b.Booking_Status,
    b.Booking_date,
    ROW_NUMBER() OVER (ORDER BY t.TicketID) as TicketNumber
FROM Ticket t
JOIN Seat s ON t.HNumber = s.HNumber AND t.SeatNumber = s.SeatNumber
JOIN Booking b ON t.BookingID = b.Booking_ID
JOIN Show sh ON b.B_shownumber = sh.show_Number
JOIN Movie m ON sh.sh_EIDR = m.EIDR
```



```
WHERE b.Booking_ID = @bookingId AND b.Customer_ID = @customerId
ORDER BY t.TicketID ASC
```

10. Load All Customer Bookings

```
SELECT * FROM Booking
WHERE Customer_ID = @id
```

11. Cancel Booking

```
UPDATE Booking
SET Booking_Status = 'Canceled'
WHERE Booking_ID = @id AND Customer_ID = @cust
```

12. Update Payment Method

```
UPDATE payment
SET Payment_Method = @method
WHERE payment_id = (
    SELECT Payment_ID FROM Booking
    WHERE Booking_ID = @id AND Customer_ID = @cust
)
```

13. Check Booking Status

```
SELECT Booking_Status
FROM Booking
WHERE Booking_ID = @id AND Customer_ID = @cust
```